Supplementary material: CON 3.1 and readcon-core

Rohit Goswami[a,b,c,1]

[a] *TurtleTech ehf., Fornhaga 24, 107, Reykjavík, Iceland*
[b] *Science Institute and Faculty of Physical Sciences, University of Iceland, VR-III, 107, Reykjavík, Iceland*
[c] *SURF B.V., Science Park 140, 1098 XG, Amsterdam, The Netherlands*

Abstract

This supplement gives the normative specification of CON 3.1. CON is a geometry format used in many codes, and readcon-core formalizes it. The supplement also states the reader rules. It then gives the test run of the release with the source lines of each step of its reader, the CON 3.1 corpus with the cross-check and the header variants, and the survey and drift files with the mechanism given to each library. The last sections give the records beside a checkpoint with what the specification of each format states, the restart protocol, the velocity unit and the identifiers, the timing method and tables, the measured versions, and the commands that rebuild the paper.

We keep the specification, the corpora and the timing records in this supplement. We name the script that writes each table. We state a reader rule as an input and the outcome that follows from it. We measure a wall time as the median of at least six repeats. We place the type 7 interquartile range next to that median. We compare each peer on the same file and the same host. We call two reads equal when their coordinates match. We count the frames a reader returns and the bytes it keeps. We draw each restored figure from a record in this directory. We leave a version deposit unnamed until the record exists. We build the library from the untagged commit named in the manuscript. That commit is not on the public main branch. We run one cell at a time on one host. We name the processor family and model, the model string, the clock, the core count, the memory, the kernel, the operating system and the compilers.

Email address: rgoswami@ieee.org (Rohit Goswami)
[1] ORCID: 0000-0002-2393-8056.

S1. Normative specification

This section is the specification of CON 3.1 as readcon-core 0.18.1 distributes it, the file `docs/orgmode/spec.org` of the untagged commit `f2cea7e8` at git blob `5c3d7869`. `scripts/gen_spec_supp.py --as-is` typesets that file, and its `--check` fails unless the typeset text matches the file. That file is byte for byte the CON 3.1 text of readcon-core 0.18.0, the first release that includes it, with its version line, its date and its 125 numbered clauses. The library test `tests/ci_gates.rs` runs `scripts/check_spec_text.sh`, which fails when the two differ. Two informative sentences of its units section round the value the parser gives the natural time unit and its difference from the value of eOn, and they are left out here. The parser reads `(amu*angstrom^2/eV)^0.5` as the binary64 number 10.180505710759414 fs, eOn hard-codes 10.1805055 fs, and their difference relative to the parsed value, computed in binary64, is $2.0702253843273332 \times 10^{-8}$ (`bench/results/velocity_case.json`). The reader it specifies is readcon-core 0.18.1. Each clause keeps its identifier, so a clause cited in the corpus manifest or an error message names the same text here. Section 6 of the manuscript measures the clauses against the reader on the conformance corpus. The specification also points to an informative parsing expression grammar (PEG), `grammar/readcon.pest`, which is not reproduced. The reader-compatibility notes and the references of the source are omitted, because they describe other programs.

Requirement words

The words MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY on this page follow RFC 2119.

Specification

Version 3.1 (major 3, minor 1)
Date 2026-10-11
Status Stable
Reference implementation readcon-core
Surface grammar `grammar/readcon.pest` (included in the appendix; not the hot path)
Clause index `docs/orgmode/clauses.tsv`, emitted by `scripts/emit_clauses.py`
In-memory API implementation notes (informative for this format)

This document is the normative CON format. CON checkpoints come from rare-event work on eOn [1, 2, 3, 4]. Each normative requirement has an id `CON-<section>.<n>`. Where this prose and the included PEG disagree, these clauses win.

CON-status.1 (MUST) A reader MUST accept the version number of every minor of major 3, including minor 0 and unknown minors.

CON-status.2 (MUST) A new implementation MUST accept version 2 and implicit version 1.

File

A file is a concatenation of frames. Extensions: `.con`, `.convel`, and the compression envelopes `.con.gz`, `.convel.gz`, `.con.zst`, `.convel.zst`.

CON-file.1 (MUST) A CON file MUST be UTF-8.

CON-file.2 (MUST NOT) A CON file MUST NOT begin with a byte-order mark.

CON-file.3 (MUST) Parsers MUST accept LF (`\n`) and CRLF (`\r\n`) line endings.

CON-file.4 (MUST) Parsers MUST ignore trailing spaces and tabs on every line.

CON-file.5 (MUST) Labels MUST be compared case-insensitively after collapsing each run of spaces to one space.

CON-file.6 (MUST NOT) Writers MUST NOT insert an extra blank line between frames.

Frame and end of file

Each frame is a 9-line header, one coordinate block per atom type, then the declared sections. There is no inter-frame marker. After a complete frame the next line is line 1 of the next frame, even when that line is empty. A stray blank therefore shifts that header and fails at its line 3.

CON-frame.1 (MUST) After the last complete frame, lines that contain only whitespace MUST be ignored.

CON-frame.2 (MUST NOT) Readers MUST NOT silently drop a partial last frame.

CON-frame.3 (MUST) One to eight header lines of a further frame MUST be reported as `IncompleteHeader`, except a type count of 0 at end of file whose lines 8 and 9 are absent.

CON-frame.4 (MUST) Fewer coordinate rows than the header declares MUST be reported as `IncompleteFrame`.

CON-frame.5 (MUST NOT) A blank line followed only by whitespace to end of file MUST NOT be read as a legacy velocity block.

CON-frame.6 (MUST) A partial declared section MUST be reported as `IncompleteSection(name)`.

CON-frame.7 (MUST) After a complete frame, including its declared sections or the legacy velocity block where that applies, the next line MUST be line 1 of the next frame, even when it is empty.

Header

Line	Content
1	Free-form generator text, preserved
2	One JSON object, or free text for version 1
3	Three lengths a, b, and c, in `units.length`
4	`alpha beta gamma` in degrees
5	Free-form text, preserved
6	Free-form text, preserved
7	Type count, integer >= 0
8	Per-type counts, empty when line 7 is 0
9	Masses in `units.mass`, empty when line 7 is 0

Line 1 and lines 5-6 have no semantics. Line 1 MAY be empty, and parsers MUST keep that empty line. Line 2 MAY be empty. An empty line 2 does not start with `{`, so the frame is implicit version 1. Angstrom and amu apply when `units` or that key is absent.

CON-header.1 (MAY) Writers MAY leave lines 5 and 6 empty.

CON-header.2 (MUST) Parsers MUST preserve line 1 and lines 5 and 6.

CON-header.3 (MUST) Line 3 and the coordinate columns MUST be read in `units.length`, and line 9 in `units.mass`. Absent `units` or an absent key means angstrom and amu.

CON-header.4 (MUST) Parsers MUST accept an empty line 1 and an empty line 2. An empty line 1 is preserved. An empty line 2 is implicit version 1.

Line 2: Metadata. Line 2 is one JSON object when the file is version 2 or newer. A line that does not start with `{` after trimming is implicit version 1. Metadata is line 2. Line numbers count from 1.

CON-line2.1 (MUST) A line 2 that starts with `{` MUST be one UTF-8 JSON object and nothing else.

CON-line2.2 (MUST) Malformed JSON on that line MUST be rejected.

CON-line2.3 (MUST) A JSON object that omits `con_spec_version` MUST be rejected.

CON-line2.4 (MUST) `con_spec_version` MUST be a JSON number, the major version, not a string.

CON-line2.5 (MUST) JSON `con_spec_version` 0 or 1 MUST be rejected as `UnsupportedVersion`.

CON-line2.6 (MUST) An unknown major MUST be rejected as `UnsupportedVersion`.

CON-line2.7 (MUST) A reader of a known major MUST accept the version number of every minor of that major, including minors it does not implement.

CON-line2.8 (MUST) Unknown keys MUST be preserved on rewrite.

CON-line2.9 (MUST) `con_spec_minor`, when present, MUST be a JSON integer >= 0.

CON-line2.10 (MUST) A missing `con_spec_minor`, or an explicit 0, MUST mean minor 0.

CON-line2.11 (MUST) `sections`, when present, MUST be an array of strings.

CON-line2.12 (MUST) `validate`, when present, MUST be a boolean.

CON-line2.13 (MUST) Reserved keys MUST use the types in this document.

Cell. Line 3 is `|a| |b| |c|`. Line 4 is `alpha angle(b,c)`, `beta angle(a,c)`, `gamma angle(a,b)`, in degrees. `a` lies along x and `b` lies in the xy plane. eOn 3.6.0 reads that placement at `client/ConFileIO.cpp:659-681` and writes it at `:531-549`. ASE `cellpar_to_cell` uses the same orientation by default (`ase/geometry/cell.py:50`, `ab_normal=(0,0,1)`). `lattice_vectors` is three row vectors in `units.length`, `[a, b, c]`.

CON-cell.1 (MUST) When `lattice_vectors` is present, readers MUST prefer it over lines 3-4.

CON-cell.2 (MUST) A writer that emits `lattice_vectors` MUST write lines 3-4 consistent with that matrix at the precision the writer is using.

CON-cell.3 (MUST) A missing `pbc` key MUST be read as periodic in all three directions.

CON-cell.4 (MUST) `pbc`, when present, MUST be a length-3 array of booleans.

CON-cell.5 (MUST) `lattice_vectors`, when present, MUST be three length-3 arrays of finite numbers.

CON-cell.6 (MUST) When `lattice_vectors` is absent, a reader MUST build

$$\begin{aligned}
\mathbf{a} &= (|a|,\ 0,\ 0) \\
\mathbf{b} &= (|b|\ \cos\gamma,\ |b|\ \sin\gamma,\ 0) \\
\mathbf{c} &= \left(|c|\ \cos\beta,\ |c|\ \frac{\cos\alpha - \cos\beta\ \cos\gamma}{\sin\gamma}, \right. \\
&\qquad \left. |c| \sqrt{1 - \cos^2\beta - \left(\frac{\cos\alpha - \cos\beta\ \cos\gamma}{\sin\gamma} \right)^2} \right)
\end{aligned} \tag{1}$$

with the angles converted from degrees.

Counts. A type count of 0 is a frame with no atoms. Lines 8 and 9 are whitespace only, no coordinate block and no section follow, `sections` is absent or empty, and the legacy velocity fallback does not apply. That frame is legal with `validate` true. A per-type count of 0 is InvalidCount when the type count is 1 or more. ASE 3.29.0 `write_eon(Atoms())` writes type count 0, an empty line 8, and an unterminated empty line 9 (`ase/io/eon.py:277-307`). `ase.io.eon.read_eon` then raises `IndexError` at `images[0]` (`ase/io/eon.py:188`). The `StopIteration` from the ninth `next()` is caught inside that function (`:177-179`). `ase.io.read(path, format="eon")` raises `StopIteration` from `ase/io/formats.py:821` (`next(_iread(...))` on a read that yields no image).

CON-count.1 (MUST) A type count of 0 MUST be a frame with zero atoms: lines 8 and 9 contain only whitespace, no coordinate block and no section block follow, `sections` is absent or empty, and the legacy velocity fallback does not apply. Any other token on line 8 or 9 MUST be `InvalidCount`.

CON-count.2 (MUST) A zero-atom frame of that form MUST be accepted in every mode, including `validate` true. `CON-validate.3` still requires the `sections` key when `validate` is true.

CON-count.3 (MUST) When the type count is 1 or more, each per-type count MUST be an integer >= 1, and any other count token MUST be `InvalidCount`.

CON-count.4 (MUST) At end of file, when line 7 is 0, missing lines 8 and 9 MUST be read as empty, and `CON-frame.3` MUST NOT report that frame as `IncompleteHeader`.

CON-count.5 (SHOULD NOT) Writers SHOULD NOT write a zero-atom frame for a consumer that predates v3.1.

CON-count.6 (MUST) The coordinate blocks MUST follow header order.

CON-count.7 (MUST) The total atom count MUST equal the sum of line 8.

CON-count.8 (MUST) Component numbers in labels MUST start at 1.

Coordinates

One block per type: a symbol line, a `Coordinates of Component N` label, then one atom line per atom of that type. Columns are whitespace-separated.

Column	Token	Meaning
1	float	x, in `units.length`
2	float	y, in `units.length`
3	float	z, in `units.length`
4	integer	Constraint flag
5	integer	Atom id, optional

Per-direction constraints (column 4). Column 4 is an integer token. Bit 0 is x, bit 1 is y, bit 2 is z. Spec 1 (no JSON on line 2): 0 is free, 1 is fixed in all three directions. Spec 2 and later: 0 through 7 are the bitmask, and 1 is x only. Writers of spec 2 and later emit 7 for all-fixed.

CON-mask.1 (MUST) Column 4 MUST be an integer token with no sign and no decimal point.

CON-mask.2 (MUST) In spec 1, readers MUST treat 0 as free and 1 as fixed on x, y, and z.

CON-mask.3 (MUST) In spec 1, any other column-4 token MUST be `InvalidFixedFlag`.

CON-mask.4 (MUST) In spec 2 and later, values 0 through 7 MUST be read as the bitmask above.

CON-mask.5 (MUST) A float spelling, a sign, or a value outside the legal set MUST be `InvalidFixedFlag` in every mode, including when `validate` is absent or false.

CON-mask.6 (MUST NOT) A spec 2 or later writer MUST NOT emit 1 to mean all-fixed.

CON-mask.7 (MUST) A spec 2 or later writer MUST emit 7 for an atom fixed on x, y, and z.

Atom index (column 5). Column 5 is the atom id. No base is implied. Grouping by type reorders atoms relative to a pre-group sequence. eOn 3.6.0 stable-sorts by ascending id when the ids are unique, and keeps file order when any id repeats (`client/ConFileIO.cpp:147-176` `matter_order`, called at `:691`). v3.4.0 has the same function.

CON-id.1 (MUST) Column 5, when present, MUST be parsed as an integer in 0 through 2^64-1, and MUST NOT be parsed through binary64.
CON-id.2 (MUST NOT) Readers MUST NOT assume that ids start at 0 or at 1.
CON-id.3 (MUST) Writers MUST emit ids that are unique within the frame.
CON-id.4 (MUST) Readers MUST accept repeated ids.
CON-id.5 (MUST) The pre-grouping order of a frame MUST be ascending column-5 id when the ids are unique, and file order otherwise.
CON-id.6 (MUST) A reader MUST expose column-5 ids unchanged.
CON-id.7 (MUST) A reader or consumer that returns atoms in pre-grouping order MUST use `CON-id.5`.
CON-id.8 (MAY) A reader MAY return atoms in file order together with their column-5 ids.
CON-id.9 (MUST) A 4-column atom line MUST use the 0-based position of that atom in the frame as its id.

Sections

A section is present exactly when its name is in `sections`. An empty `sections` array means no section follows, and it disables the legacy velocity fallback. With no `sections` key, one blank line that begins a complete velocity block is a velocity section. A blank line that does not begin that block is line 1 of the next frame. That fallback is the only undeclared section, and `CON-frame.5` limits it. Minor 1 reserves the names in the table below. A vector row has arity 3 and a scalar row has arity 1. Those integers count value fields only.

Section declaration.

CON-section.1 (MUST) On disk, a section MUST be treated as present exactly when its name is listed in `sections`, aside from the legacy velocity fallback.

CON-section.2 (MUST) Exactly one blank line MUST precede each section block.

CON-section.3 (MUST NOT) Writers MUST NOT put a blank line between the component blocks inside a section.

CON-section.4 (MUST) Each component block MUST be a symbol line, a label, and one data row per atom of that type.

CON-section.5 (MUST) The per-atom energy section name MUST be `energies`.

CON-section.6 (MUST NOT) `atom_energies` MUST NOT be used as a section name.

CON-section.7 (MUST) An unknown section name MUST be reported as `UnknownSection` when the frame minor is not newer than the minor this reader implements.

CON-section.8 (MAY) When the frame minor is newer than the reader implements, the reader MAY skip an unknown section whose value-field count is given by `section_arity`.

CON-section.9 (MUST) If that minor is newer and `section_arity` does not give the value-field count, the reader MUST report `UnknownSection`.

CON-section.10 (MUST) Adding a reserved section name MUST increase `con_spec_minor`.

CON-section.11 (MUST NOT) Adding a reserved section name MUST NOT require a new major.

CON-section.12 (MUST) `section_arity`, when present, MUST be an object mapping section names to integers >= 1, each the number of value fields on a data row, excluding the flag and the optional id.

CON-section.13 (MUST) An entry for a reserved section MUST equal 3 for a vector row and 1 for a scalar row, else `InvalidVectorLength`.

CON-section.14 (MUST) A data row of an unknown section MUST hold that arity of value fields, then the flag, and an optional id, so the row has arity+1 or arity+2 tokens.

CON-section.15 (MUST) The label of any section MUST be its name compared under `CON-file.5`, followed by `of Component N`.

CON-section.16 (MUST) A writer of minor 2 or later MUST list in `section_arity` every section it emits that minor 1 does not reserve.

Name	Label	Row
`velocities`	`Velocities of Component N`	`vx vy vz` flag id
`forces`	`Forces of Component N`	`fx fy fz` flag id
`energies`	`Energies of Component N`	energy flag id
`charges`	`Charges of Component N`	charge flag id
`spins`	`Spins of Component N`	spin flag id
`magmoms`	`Magmoms of Component N`	`mx my mz` flag id
`displacements`	`Displacements of Component N`	`dx dy dz` flag id, `units.length`
`spreads`	`Spreads of Component N`	`sx sy sz` flag id, `units.length`, each >= 0

Identity columns repeat the coordinate flag and id. 0 in a `spreads` row is a classical point.

CON-section.17 (MAY) When `validate` is not true, readers MAY ignore an identity-column mismatch after the row has the right number of fields.

CON-section.18 (SHOULD) The `energy` metadata value SHOULD equal the sum of the `energies` rows.

CON-section.19 (MUST NOT) A mismatch between that sum and `energy` MUST NOT reject the frame.

CON-section.20 (MUST) Each `spreads` value MUST be finite and >= 0.

Validation mode.

CON-validate.1 (MUST) `validate` true MUST add the checks in `CON-validate.4`.

CON-validate.2 (MUST NOT) `validate` true MUST NOT relax a check that already applies in every mode.

CON-validate.3 (MUST) When `validate` is true, `sections` MUST be present, and an empty array is legal.

CON-validate.4 (MUST) In that mode, readers MUST reject a bad reserved-key type, a non-positive cell length, an angle outside (0, 180) degrees, a non-positive mass, a coordinate label that does not match, an unrecognized symbol other than `X`, a section label or symbol mismatch, or an identity column that does not match the coordinate row, as `ValidationError`.

Frame topology (bonds). `bonds` is optional frame topology, not a section. Each entry is `[i, j]` or `{"i": i, "j": j}` with an optional integer `order`. Indices are 0-based into coordinate-block order, not column-5 ids. This key is legal on major 3. It does not change the major.

CON-bonds.1 (MUST NOT) `bonds` MUST NOT appear in `sections`.
CON-bonds.2 (MUST) With `validate` true, a malformed `bonds` entry MUST be rejected.
CON-bonds.3 (MUST) With `validate` false, unknown `bonds` shapes MUST be preserved.

Units (required)

`units` maps a dimension to a unit expression: names combined with `*`, `/`, `^`, and parentheses. Version 3 requires `length` and `energy`. Velocity is length/time taken from `units`.

The natural time unit used by eOn and by ASE, verified by an eOn 3.6.0 round trip, is written exactly `(amu*angstrom^2/eV)^0.5`. That expression is a time. eOn hard-codes 10.1805055 fs (`include/eon/ParametersOptions.h:26`). A writer that canonicalises the expression by inserting spaces is not using the pinned spelling. Names below are the ones `src/units.rs` at f09b1af accepts for length, energy, mass, and time, compared case-insensitively.

CON-units.1 (MUST) Major 3 MUST include `units.length` and `units.energy` as non-empty strings of the matching dimension.
CON-units.2 (MUST) Optional `mass`, `time`, `velocity`, and `force` strings, when present, MUST have the matching dimension.
CON-units.3 (MUST) A writer of minor 1 or later MUST write `units.time` whenever the frame has a velocities section.
CON-units.4 (MUST NOT) A writer MUST NOT insert `time: fs` silently when the frame does not carry a time unit.
CON-units.5 (MUST) The natural unit MUST be written with the exact spelling `(amu*angstrom^2/eV)^0.5`.
CON-units.6 (MUST NOT) Version 1 MUST NOT carry a `units` object.
CON-units.7 (MAY) Version 2 MAY carry `units`.
CON-units.8 (MUST) In versions 1, 2, and 3.0, velocities MUST be read as `units.length` per the natural time unit, and length is angstrom when `units.length` is absent.

CON-units.9 (MUST NOT) Any `units.time` on versions 1, 2, and 3.0 MUST NOT be trusted.

CON-units.10 (MUST) From minor 1 onward, `units.time` MUST be authoritative for velocities.

CON-units.11 (MUST) A frame of minor 1 or later with a velocities section and no `units.time` MUST be rejected with `InvalidUnits`.

CON-units.12 (MUST) A writer of minor 1 or later asked to write velocities with no time unit MUST fail with `InvalidUnits`.

CON-units.13 (MUST) An invalid unit expression on write MUST be reported as `InvalidUnits`.

CON-units.14 (MUST NOT) Writers MUST NOT replace an invalid unit with a default.

CON-units.15 (MAY) A consumer MAY apply an explicit velocity time-unit override when a file breaks `CON-units.8` through `CON-units.10`.

CON-units.16 (MUST) A reader MUST accept at least these names case-insensitively: length m, meter, metre, angstrom, a, å, nm, nanometer, bohr, a0; energy j, joule, ev, mev, hartree, ha, kcal, kj; mass kg, kilogram, u, amu, dalton, da; time s, second, fs, femtosecond, ps, picosecond, ns, nanosecond; count mol.

Minor 0 is the v3.0 rule because the producers of those velocity sections wrote natural-unit numbers under a `fs` label.

Numbers

CON-num.1 (MUST) Readers MUST reject NaN, Inf, Infinity, and the signed and mixed-case spellings of those words, in every numeric field and in every mode, as `NonFiniteValue`.

CON-num.2 (MUST NOT) `validate` true MUST NOT be required before a reader applies `CON-num.1`.

CON-num.3 (MUST NOT) `validate` true MUST NOT accept a non-finite value.

CON-num.4 (MUST NOT) The C ABI absent-value NaN sentinel MUST NOT be written to a file. It is specified in the implementation notes.

CON-num.5 (MUST) Readers MUST convert a finite decimal token to binary64 with correct rounding.

CON-num.6 (MUST) A conforming writer in its default mode MUST write each finite float so that parsing it returns the same binary64 (`FloatFormat::RoundTrip`).

CON-num.7 (MUST) Fixed-decimal output MUST be used only when the writer is asked for it, and it need not round-trip.

CON-num.8 (MUST) A token that is not a decimal number and not a non-finite spelling MUST be `InvalidNumberFormat`, except where `CON-error.3` names another class.

CON-num.9 (MUST) A row with the wrong number of fields MUST be `InvalidVectorLength`.

eOn `.convel` output at 6 digits after the decimal (`kConvelPrecision` in `client/ConFileIO.cpp:43`) is fixed-decimal and lossy. The kinetic-energy loss for that rounding was measured on one 336-atom Pt slab at 300 K (one Maxwell-Boltzmann draw) as 5.8e-7 relative.

A float token is an optional `+` or `-`, a significand with at least one digit, an optional fraction (a leading dot is allowed), and an optional exponent. An integer token is one or more ASCII digits and nothing else.

Compression

CON-compress.1 (MUST) A gzip envelope MUST expose every member of a multi-member file, in order, as CON text.

CON-compress.2 (MUST) A zstd envelope MUST expose every frame in the stream as CON text.

CON-compress.3 (SHOULD) Readers SHOULD detect gzip and zstd by magic bytes, not only by the extension.

Errors

Line numbers count from 1. Metadata failures are line 2. `IncompleteSection` carries the section name. A reader that stops at the first error MUST use the lowest line number, so threads cannot change which error is reported.

Class	When
`MissingSpecVersion`	Line 2 starts with `{` but omits `con_spec_version`.
`InvalidMetadataJson`	Line 2 is malformed JSON, a reserved key has the wrong type, or `validate` true omits `sections`.
`NonFiniteValue`	NaN, Inf, or a spelling of either, any mode.
`InvalidFixedFlag`	Column 4 is not a legal integer flag, any mode.
`InvalidCount`	A count field is not an unsigned integer, or a per-type count is 0 when the type count is 1 or more. A type count of 0 is a legal zero-atom frame.
`InvalidUnits`	A unit expression is missing, mistyped, or the wrong dimension. A minor of 1 or later with velocities and no `units.time` is this class on read and on write.
`IncompleteHeader`	A started frame has too few header lines, other than `CON-count.4`.
`IncompleteFrame`	A coordinate block is shorter than declared.
`IncompleteSection(name)`	A declared section is missing or short.
`InvalidNumberFormat`	Column 5 or another field is not a decimal number.
`InvalidVectorLength`	A row has the wrong number of fields, or a reserved arity is not 3 for a vector or 1 for a scalar. An unknown-section row has that many value fields, then a flag, and an optional id.
`UnknownSection`	A section name is not known and cannot be skipped.
`UnsupportedVersion`	Unknown major, or JSON major 0 or 1.
`ValidationError`	A `validate` true check failed. It MUST NOT relax `NonFiniteValue` or `InvalidFixedFlag`. A legal zero-atom frame is accepted.

CON-error.1 (MUST) Each class in the table MUST be distinguishable by a caller.

CON-error.2 (MUST) Reported line numbers MUST count from 1, with metadata on line 2.

CON-error.3 (MUST) A count field that is not an unsigned integer, including a sign, a decimal point, or `-1`, MUST be `InvalidCount`. Column 4 MUST be `InvalidFixedFlag`. Column 5 and every other such field MUST be `InvalidNumberFormat`.

CON-error.4 (MUST) A reader that stops at an error MUST report the one with the lowest line number.

Version history

Version	Date	Change
1	original	eOn text. Column 5 present. Column 4 is 0 or 1. No JSON.
2	2026-03-25	JSON line 2, ids, bit masks, declared sections.
3.0	2026-06-27	`units.length` and `units.energy` MUST be present. Minor absent.
3.1	2026-10-11	Minor 1. `units.time` when velocities are present. Natural-unit rule.

Normative changes on 2026-10-11, by decision:

- Non-finite numbers are rejected in every mode (`D1`).
- A partial last frame is an error. Trailing whitespace is not a frame. A blank line and then only whitespace is not a velocity block (`D2`).
- A section is present exactly when its name is in `sections`. Builder clearing is not a format rule (`D3`).
- A type count of 0 with empty lines 8 and 9 is a legal zero-atom frame. A zero per-type count when the type count is >= 1 is `InvalidCount`. D4 revised after checking eOn 3.6.0 (`D4`).
- One blank line before a section, and none between its component blocks (`D5`).
- The scalar energy section is `energies` (`D6`).
- `bonds` is specified for major 3 (`D7`).
- Column 4 is an integer flag, with spec-1 and spec-2 meanings (`D8`).
- Column 5 is an integer id. Unique ids define ascending pre-grouping order. A repeated id keeps file order (`D9`).
- Velocity units follow the version rules, and the natural spelling is pinned (`D10`).
- Invalid units fail the writer (`D11`).
- Lines 3-4 and `lattice_vectors` follow the cell convention. Missing `pbc` is fully periodic (`D12`).
- Default float text round-trips binary64. Fixed decimals are opt-in (`D13`).
- Unknown keys are kept. Unknown sections of a newer minor MAY be skipped by `section_arity` (`D14`).
- The major stays 3. `con_spec_minor` is 1. A same-major reader accepts every minor's version number (`D15`).
- Gzip members and zstd frames are normative envelopes (`D16`).

- Line 2 is one JSON line. Trailing space is ignored. Labels collapse spaces (D17).

`con_spec_minor` is the minor of that major. `ConFrameBuilder` stamps `con_spec_minor` 1 on every spec 3 frame it builds when the caller has not set the key. `set_con_spec_minor` (Rust, `rkr_frame_builder_set_con_spec_minor`, and the C++ method of the same name) overrides that value before `build`. The writer MUST NOT invent or replace `con_spec_minor` (D15).

The media type `chemical/x-con` is informative, as is `text/plain; charset=utf-8`. CON has no magic bytes. Identify a file by extension and by this header.

Examples

Version 3.1. Two types, `lattice_vectors`, `pbc`, masks 0, 3, and 7, ids out of order, velocities with `units.time`, and forces. Ids 2, 0, 1 appear in that file order. Mask 3 is x and y. Mask 7 is all three.

```
Generated by readcon v3.1 example
{"con_spec_version":3,"con_spec_minor":1,"units":{"length":"angstrom","energy":"eV","mass
    ↪ ":"amu","time":"(amu*angstrom^2/eV)^0.5"},"pbc":[true,true,false],"lattice_vectors
    ↪ ":[[10.0,0.0,0.0],[0.0,10.0,0.0],[0.0,0.0,20.0]],"sections":["velocities","forces"]}
    ↪
10.0 10.0 20.0
90.0 90.0 90.0
0 0
0 0 0
2
2 1
195.084 1.00794
Pt
Coordinates of Component 1
0.0 0.0 0.0 0 2
5.0 5.0 0.0 3 0
H
Coordinates of Component 2
5.0 5.0 10.0 7 1

Pt
Velocities of Component 1
0.01 0.0 0.0 0 2
0.0 0.02 0.0 3 0
H
Velocities of Component 2
0.0 0.0 0.03 7 1

Pt
Forces of Component 1
0.1 0.0 0.0 0 2
0.0 -0.2 0.0 3 0
H
```

```
Forces of Component 2
0.0 0.0 0.3 7 1
```

The same frame as a version 1 reader sees it. Spec 1 cannot store masks 3 or 7, JSON, or sections. The projection below uses column 4 values 0 and 1 only. Mask 3 and mask 7 both become 1, meaning all-fixed. Velocities and forces are omitted because a version 1 reader that treats a blank line as the next header cannot return them. ASE `read_eon` is that reader.

```
Generated by readcon v3.1 example
0.0000 TIME
10.0 10.0 20.0
90.0 90.0 90.0
0 0
0 0 0
2
2 1
195.084 1.00794
Pt
Coordinates of Component 1
0.0 0.0 0.0 0 2
5.0 5.0 0.0 1 0
H
Coordinates of Component 2
5.0 5.0 10.0 1 1
```

Zero atoms. Nine header lines. Line 7 is 0. Lines 8 and 9 are empty. No coordinate block and no section follow.

```
zero-atom frame
{"con_spec_version":3,"con_spec_minor":1,"units":{"length":"angstrom","energy":"eV"}}
10.0 10.0 10.0
90.0 90.0 90.0


0
```

Token grammar

CON-token.1 (MUST) Integer tokens MUST match one or more ASCII digits, with no sign and no decimal point.

CON-token.2 (MUST) Float tokens MUST be finite decimals. A leading sign, a leading dot, and an exponent are allowed. Non-finite spellings are `CON-num.1`, not floats.

CON-token.3 (MUST) The separators inside a line MUST be spaces or tabs.

Table S1: Conformance corpus of readcon-core 0.18.1 by the verdict that a conforming reader returns on each group of files. The counts come from reading every file of the corpus with that release.

Category	Files	What a reader must do
Header and units	2	Accept a version-2 header and a version-3 units line.
Column-4 masks	5	Accept version-1 full freeze and version-2 per-component flags.
Empty frame	3	Accept a type count of zero.
Empty header lines	4	Accept an empty line 1 and an empty line 2.
Illegal	8	Reject the file.

CON-token.4 (MUST) The included PEG is informative surface syntax. A reader MUST follow this document when the PEG allows a token these clauses reject.

Table S2: Seven XYZ reader paths on seven three-atom files. Five paths from four libraries document extended XYZ; the libAtoms package is one library with a C and a Python parser. A reader with an asterisk is a plain XYZ reader whose documentation does not claim extended XYZ. Each file departs from the reference frame in one respect. ✓: each position the binary64 value nearest its decimal, periodicity as written, mask and ids returned as written. E: the read failed. u: coordinates at most 2 units in the last place from the binary64 values nearest their decimals. f: single-precision coordinates. n: a coordinate returned as NaN. p: periodicity differs from the file, or from the grammar's default for a file without the key. b: the mask came back with different booleans. m, i: the result has no `move_mask` or no `id` column. The grammar defines neither name and requires no reader to return either column. Both follow ASE's layout, so m and i mark a reader outside that convention.

File	ASE	libAtoms C	libAtoms Py	chemfiles	OVITO	MDAnalysis*	Open Babel*
reference frame	✓	✓	b	u	✓	fmi	mi
no pbc key	✓	✓	b	u	✓	fmi	mi
plain comment line	✓	p	E	u	✓	f	✓
Fortran exponent `6.0d0`	E	✓	nb	E	✓	E	mi
`True`/`False`	✓	✓	b	u	E	fmi	mi
quoted value with `=`	✓	✓	b	u	✓	fmi	mi
`units` key	✓	✓	b	u	✓	fmi	mi

Table S3: Fields returned when the Pt(100) frame of Listing 1 is written by one library and read back by the same library or another. The bottom layer is held in every component, the adatom in z, the identifiers are not the row order, and the velocities are in the natural unit. Each library stores each field through the mechanism its own documentation gives for it. A returned field is fixed by the format, kept or declared, when the format's own specification defines the place it was stored in, and a convention when only a library's documented layout defines that place, so only readers that share the layout get it back. Declared means that the reader took the velocity unit from the file and that the velocities agree after conversion with it, and no unit means that the file names no time unit. Label means that the identifier came back inside the CIF site label, which CIF defines as a unique site name. The H5MD file with a private mask dataset has the H5MD root group and version and keeps the mask in a dataset that H5MD does not define. Shift is the largest coordinate change when a coordinate differs in any bit, with the magnitude of that coordinate and the change in units in the last place (ulp). Periodicity is the flags of the slab, periodic in x and y and open in z, after the read, and version is the version the written file carries.

Format	Writer	Reader	Mask	Identifier	Time unit	Shift (Å)	at (Å), ulp	Periodicity	Version
CON 3.1	readcon-core	readcon-core	kept	kept	declared	exact	–	kept	3.1
extended XYZ	ASE	ASE	convention	convention	no unit	4.7×10^{-9}	7.96, 5.3×10^{6}	kept	none
extended XYZ	ASE	chemfiles	convention	convention	no unit	4.7×10^{-9}	7.96, 5.3×10^{6}	kept	none
extended XYZ	ASE	OVITO	convention	convention	no unit	4.7×10^{-9}	7.96, 5.3×10^{6}	kept	none
extended XYZ	ASE	MDAnalysis	lost	lost	lost	2.2×10^{-7}	7.96, 2.4×10^{8}	not returned	none
extended XYZ	chemfiles	chemfiles	convention	convention	lost	4.9×10^{-6}	7.96, 5.5×10^{9}	kept	none
extended XYZ	chemfiles	ASE	convention	convention	lost	4.9×10^{-6}	7.96, 5.5×10^{9}	kept	none
extended XYZ	OVITO	OVITO	convention	convention	no unit	8.9×10^{-16}	6.00, 1	all periodic	none
extended XYZ	OVITO	ASE	convention	convention	no unit	exact	–	all periodic	none
ASE trajectory	ASE	ASE	convention	convention	no unit	exact	–	convention	ULM 3, v1
POSCAR	ASE, FixScaled	ASE	kept	lost	no unit	4.5×10^{-17}	6.1×10^{-4}, 411	all periodic	none
H5MD	MDAnalysis	MDAnalysis	lost	lost	declared	2.2×10^{-7}	7.96, 2.4×10^{8}	not returned	1.1
H5MD, private mask dataset	h5py	h5py	convention	kept	declared	exact	–	kept	1.1
XYZ	Open Babel	Open Babel	lost	lost	lost	4.9×10^{-6}	7.96, 5.5×10^{9}	not returned	none
CIF	Open Babel	Open Babel	lost	label	lost	6.7×10^{-5}	7.96, 7.6×10^{10}	all periodic	none

S2. Reader rules

Tables S4, S5 and S6 collect the edge cases a reader applies, stated as input and outcome. Each row restates clauses of Section S1.

Table S4: Reader rules for masks, identifiers, versions, metadata and sections.

Input	Rule
Column 4 outside the legal set, a sign or a float spelling	Illegal flag, in every mode.
Version 1 column 4 equal to 1	Fixed on x, y and z.
Version 1 column 4 other than 0 or 1	Illegal flag.
Atom row without column 5	Identifier is the row's zero-based position in the frame, row by row.
Repeated identifier	Accepted; file order is kept.
Unique identifiers	Ascending identifier is the order before grouping by type.
Line 2 starts with `{`	JSON header; otherwise version 1.
`con_spec_version` not an integer, such as `3.5` or `"3"`	Invalid metadata.
JSON object without `con_spec_version`	Rejected.
JSON major other than 2 or 3, a negative one included	Unsupported version.
Later minor of a known major, of any size	Accepted.
`con_spec_minor` not a non-negative JSON integer	Invalid metadata.
Major 3 without `units.length` or `units.energy`	Rejected.
Unknown metadata key	Kept on rewrite, each number with the digits the file gave it.
Unknown section at the reader's minor or earlier	Rejected.
Unknown section at a later minor, width w in `section_arity`	Skipped; each row has $w+1$ or $w+2$ tokens, counted and not parsed.
Symbol line of a skipped section that differs from the coordinate block	Validation error with `validate` true; not checked otherwise.
Skipped section, then rewritten	Rows dropped.
Unknown section at a later minor, no width	Rejected.
Section label not *Name* `of Component` N	Incomplete section; validation error with `validate` true.
Component number not in plain decimal, such as `01` or `+1`	Label mismatch.
Section blocks	Read in the order `sections` lists.

Table S5: Rules for velocities, the cell and numbers.

Input	Rule
Velocity block at major 3, minor 1 or later, without `units.time`	Rejected.
Velocity block at major 2, any minor, without `units.time`	Accepted.
Velocity block of version 1, 2 or 3.0	Natural unit, whatever time `units.time` names; the label is kept and not applied.
`units.time` that is no time, such as `banana`	Invalid units, in every version.
No `sections` key, blank line before a complete velocity block	Velocity section.
No `sections` key, any other blank line after the coordinates	Line 1 of the next frame.
Cell length not positive, or angle outside the open interval to 180°	Rejected with `validate` true; not checked otherwise.
Angles whose computed radicand $\hat{\rho}$ of Eq. (1) is at most -1×10^{-12}, without `lattice_vectors`	Validation error, in every mode.
Angles whose computed radicand lies between -1×10^{-12} and zero	Flat cell with $c_z = 0$.
`lattice_vectors` in the header	The cell; lines 3 and 4 build none.
Coordinate label other than `Coordinates of Component` N	Validation error, in every mode.
`nan`, `inf` or `infinity` in any case and sign, or a decimal past binary64 such as `1e999`	Non-finite value, in every mode.
Token that begins like those spellings and is none of them, such as `NaN.5`	Invalid number format.
Form feed, vertical tab, lone CR or no-break space inside a line	Part of the field it touches.

Table S6: Rules for framing and compression.

Input	Rule
Type count of zero	Legal empty frame when lines 8 and 9 hold only spaces and tabs and `sections` is absent or empty.
Per-type count of zero under a type count of 1 or more	Rejected.
Short header or short section	Rejected.
Cut that leaves a legal file, such as a last row without its identifier	Read as that file; CON has no trailer.
Empty first header line	User line, kept.
Empty second header line	Version 1.
Byte-order mark at the start of the file	Rejected.
Byte-order mark at the start of a later frame	Text of line 1 of that frame.
Byte that is not UTF-8	Rejected at the line that contains it, on every read path.
Errors on more than one line	The lowest line is reported.
Line of Unicode white space only, such as a form feed, a vertical tab, a no-break space or U+2003	Blank line.
After the last frame, one blank line followed only by spaces, tabs, form feeds and line ends	Ignored.
After the last frame, a blank line and then a line of other Unicode white space, such as U+2003	A further frame, rejected as a short header.
File of zero bytes, or of one blank line followed only by spaces, tabs, form feeds and line ends	Zero frames.
Several gzip members	Read in order.
zstd input to a build without the `zstd` feature	Refused as unsupported zstd input.

S3. Test run and source lines

Listing S1 runs the `rust_usage` example of the release on the file of Listing 1 in a clone of the bundle. The program prints the cell, the counts and the first five atoms with their fixed flags, identifiers and velocities. `scripts/test_run.py` then runs the same program on that file with line 4 replaced by the two variants of Section 2, angles of 30, 60 and 90 degrees and of 10, 10 and 170 degrees, each as written and with `"validate": true` in line 2. In both modes the program reads the flat cell and prints its angles. In both modes it stops on the impossible cell with the error `cell angles do not form a real cell` in line 4. `paper/cpc/data/test_run.json` keeps the commands, the toolchain and what the program printed for each of the five files.

Listing S1: Test run. The example program of readcon-core 0.18.1, built with rustc 1.88.0, reads Listing 1 and prints its cell, its counts and its first five atoms with their fixed flags, identifiers and velocities.

```
$ git clone readcon-core.bundle readcon-core
$ cd readcon-core
$ git checkout 7b4b2563
$ tar -xzf ../paper-cpc.tar.gz
$ cargo run --release --locked --example rust_usage -- paper/cpc/src/
    figures/generated/survey_frame.con
Parsed 1 frame(s) from 'paper/cpc/src/figures/generated/survey_frame.con'

--- Frame 1 ---
  Cell:   [5.5437, 5.5437, 13.9600]
  Angles: [90.0000, 90.0000, 90.0000]
  Atom types: 1
  Total atoms: 9
  Has velocities: true
  Atom 0: Pt (1.3865, 1.3859, 5.9992) fixed=true id=17 vel=(0.000000,
    0.000000, 0.000000)
  Atom 1: Pt (4.1585, 1.3861, 5.9998) fixed=true id=3 vel=(0.000000,
    0.000000, 0.000000)
  Atom 2: Pt (1.3861, 4.1582, 6.0001) fixed=true id=11 vel=(0.000000,
    0.000000, 0.000000)
  Atom 3: Pt (4.1575, 4.1578, 6.0010) fixed=true id=5 vel=(0.000000,
    0.000000, 0.000000)
  Atom 4: Pt (-0.0003, 0.0006, 7.9599) fixed=false id=23 vel=(-0.004419,
    -0.003123, 0.005325)
  ... and 4 more atoms
```

Table S7 gives the source lines of each step of Algorithm 1 and of the other read and write paths of Section 4.

Table S7: Source lines of the reader and the writers, in the files under `src` of the untagged commit `f2cea7e8`. `scripts/src_anchors.py` finds each range from the anchors of `paper/cpc/data/src_anchors.toml` and fails when an anchor matches no line or more than one.

Step	Source lines
Decompress every gzip member	`compression.rs:224`
Decompress zstd input, or refuse it without the feature	`compression.rs:226`
Read a short plain file whole, map a longer one	`compression.rs:168-176`
One thread or a pool, by the thread count and the length	`iterators.rs:1248-1277`
Walk the frame boundaries by the counts	`iterators.rs:1872-1915`
Parse the frames in a pool, in file order	`iterators.rs:1949-1954, 2391-2395`
Byte-order mark and the JSON test of line 2	`iterators.rs:925`
Header lines and the version rule of Eq. (6)	`parser.rs:1266-1503`
Radicand of the cell	`parser.rs:1752-1766`
Symbol line, label and rows of each type	`parser.rs:1642-1702`
Coordinate rows	`parser.rs:1713-1750`
Finite numbers and masks	`parser.rs:299-314, 337-359`
Legacy velocity block	`iterators.rs:366-403`
Declared sections and skipped sections	`parser.rs:2165-2277, 1891-1929`
Velocity section without a time unit	`parser.rs:2294-2318`
Path writer: temporary file, rename and removal	`frame_index.rs:388-399`
All frames at the offsets of a frame index	`iterators.rs:2115-2135`
Trajectory open, with or without a written index	`trajectory.rs:24-33`
Binary image: layout check and copy	`conbin.rs:246-262, 312-339`
Binary image: payload-offset check	`conbin.rs:293-302`

S4. The CON 3.1 corpus, the cross-check and the header variants

This section describes the second corpus of Section 6, the cross-check of both corpora and the seven headers of Table S11. `paper/cpc/data/corpus31` contains the 143 files of that corpus, which `scripts/make_corpus31.py` writes byte for byte, and its `manifest.toml` names for each file the outcome, the clauses that decide it and the basis of that outcome in the released text. `paper/cpc/data/corpus31-results.json` is the record of `scripts/corpus31_results.py`: the outcome of every file at one, two and eight threads and the default, from the `corpus31_report` example of readcon-core, and the build that read the files. `scripts/corpus_paths.py` writes `paper/cpc/data/corpus_paths.json`. It reads the 165 files of both corpora through the string, file, mapped, gzip and iterator reads and the frame count of the readcon module and of the C library. The C library is read in its default build and built with `parallel`, as the release archives build it, and the module also reads the plain and the gzip file through its stream reader, `iter_con(stream=True)`. Every file takes 57 reads, except that the 3 files that are not UTF-8 take no `read_con_string` read. All 9,393 of the 9,393 reads return the outcome of the manifest entry of their file, with its frame count or error class (Table S9). The record keeps the verdict of every read with the builds that made them. `paper/cpc/data/every_path.json` is the record of `scripts/every_path_count.py`, which runs the 8 reader-path test files of readcon-core with the `zstd` and `parallel` features, 47 tests in all, and counts the 694 inputs that passed every path, each logged once as given and once padded. Table S8 lists the 48 entries whose outcome no clause decides by name, each with the reading that its manifest entry records and that readcon-core follows. Table S10 is the run of `scripts/corpus_crosscheck.py` over both corpora, beside the independent reader `scripts/indep_reader.py`; `paper/cpc/data/corpus_crosscheck.json` and `paper/cpc/data/corpus_crosscheck.csv` record the outcome of every reader on every file.

`paper/cpc/data/spec/mutant_differential.json` is the record of `scripts/mutant_differential.py`. A case takes one of the 132 files of `paper/cpc/data/corpus31` under 4,000 bytes and makes one to three edits at random places: it inserts a token, deletes one to eight characters or replaces one character with a token. The tokens are whitespace and line ends, digits, signs and JSON punctuation, component headers, keys of line 2, element symbols and a whole velocity block, and the record lists them.

`random.Random` draws the file, the edits and the tokens from the seed. Each of the 7 seeds gives the same 3,000 cases on every run, and `--check` reruns them. The comparison covers the verdicts and the frame counts or error classes and leaves out the lines, since the exceptions of the readcon module carry none. The record still keeps the error line of each reader. Each group has a variant of the independent reader that follows readcon-core on one rule. A differing case belongs to the first group whose variant returns the verdict and class of readcon-core. All 15 differing cases fall in the count-line group. The record leaves 0 cases unexplained. The first variant is a count line with the wrong number of tokens and a token that is not an unsigned integer, so it breaks CON-error.3 and CON-num.9 on one line; CON-error.4 orders errors by line alone, and readcon-core checks the number of tokens first, the independent reader each token first. The second variant is line 2 with a number such as `1e999`, which the JSON grammar of Request for Comments (RFC) 8259 admits and binary64 cannot represent. The independent reader rejects such a number anywhere in line 2, and readcon-core keeps it in an unreserved key and reads on; no clause requires either reading. The third variant is a last byte that is a carriage return, of which CON-file.3 states nothing: readcon-core reads it as a line end, so the input ends in trailing whitespace, and the independent reader takes it for line 1 of a further frame, which ends before its line 2. `paper/cpc/data/spec/clause_coverage.json`, the record of `scripts/clause_coverage.py`, lists for each of the 125 clauses the corpus files whose manifest entries name it, the probes whose rules name it, the tests of the library whose names or documentation name it and whether the independent reader names it. No file, probe or test names CON-count.5, a SHOULD NOT on writing for a given consumer, since no writer of readcon-core takes a consumer or a target version; CON-section.10 and CON-section.11, which govern how a later revision adds a reserved section name; and CON-section.18, a SHOULD on the energies a producer stores, whose reader side CON-section.19, accepting a frame whose energy differs from the sum of its energies rows, `tests/spec_clause_gaps.rs::con_section_19_energy_sum_mismatch_does_not_reject_the_frame` reads.

The test `differential_corpus_mutations_agree_across_entry_points` of `src/iterators.rs` draws 140 cases, each conformance file and each test file with one of four edits: a form feed, a no-break space, two blank lines, or a further section at its end. Each case is read from a string and from bytes at one to sixteen threads, by the stream reader at read sizes of 1

and 61 bytes and 1 MiB, and by the column reader with the positions alone and with every section, from bytes and streamed from the file. It is then written to a plain and a gzip file, and each file is read again at the same thread counts, with the stream reader and through the indexed trajectory. Every read returns the frame count, the coordinates and the section presence of the one-thread string read or stops with its error at the same line, and the trajectory refuses to open a file only when the string read rejects the case. One case in every 305 is also padded with blank lines past the mapping threshold and read as a mapped file at one to sixteen threads, and each read returns the outcome of a one-thread read of the padded text. `cargo test --release --lib differential_corpus` runs it.

Table S8: Entries of the CON 3.1 corpus whose outcome no clause decides by name, each with the reading that its manifest entry records.

File	Outcome; basis	Clauses	Reading
`v2_minor1_velocity_no_time.convel`	accept, 1 frame; open	CON-units.8, CON-units.11	Major 2 with con_spec_minor 1. CON-units.8 reads version 2 velocities in the natural time unit, and CON-units.11 rejects a frame of minor 1 or later without units.time. The reader takes a minor to refine major 3 only, so it reads the velocities in the natural time unit.
`v31_units_compound.convel`	accept, 1 frame; open	CON-units.1, CON-units.2, CON-units.16	Products, quotients and parentheses of the matching dimensions (Units). The energy is written kJ / mol; the text does not say whether white space may stand inside an expression, and the reader removes it.
`v32_unknown_section_arity.con`	accept, 1 frame; optional	CON-line2.7, CON-section.8, CON-section.14	Frame minor 2 is newer than the reader's minor 1 and section_arity gives stress 6 value fields, so the reader may skip the stress section, and it does.

File	Outcome; basis	Clauses	Reading
`v33_unknown_section_arity.con`	accept, 1 frame; optional	CON-line2.7, CON-section.8, CON-section.14	Frame minor 3 is newer than the reader's minor 1 and section_arity gives stress 6 value fields, so the reader may skip the stress section, and it does.
`v31_two_blanks_before_section.convel`	`IncompleteSection`, line 16; class-open	CON-section.2, CON-section.4, CON-section.15	Two blank lines before the velocities section: the first opens it, the second is its symbol line, and the line Pt fails as its label. Outside validate true no clause names the class of a wrong label; the reader reports the declared section as missing.
`v31_unicode_blank_before_section.convel`	accept, 1 frame; open	CON-section.2, CON-file.4	The line before the velocities section holds U+2003 only. The reader takes a line of Unicode White_Space characters to be blank before a section or a legacy velocity block. The ignored end of the file is one such line, then lines of spaces, tabs, form feeds and carriage returns only. The text does not define whitespace or blank otherwise. So the line is blank and opens the declared section.
`v31_spreads_negative.con`	`ValidationError`, line 17; class-open	CON-section.20	Each spreads value must be >= 0 in every mode; the clause names no class, and the reader reports ValidationError.
`v31_section_label_mismatch.convel`	`IncompleteSection`, line 16; class-open	CON-section.15	The section label reads Velocity of Component 1 without validate. CON-section.15 applies in every mode and names no class; the reader reports the declared section as missing.

File	Outcome; basis	Clauses	Reading
`v31_label_velocities_no_index.convel`	`IncompleteSection`, line 16; class-open	CON-section.15	The label of component 1 of the velocities section reads Velocities of Component; the reader reports the declared section as missing.
`v31_label_velocities_xyz.convel`	`IncompleteSection`, line 16; class-open	CON-section.15	The label of component 1 of the velocities section reads Velocities of Component xyz; the reader reports the declared section as missing.
`v31_label_velocities_1_xyz.convel`	`IncompleteSection`, line 16; class-open	CON-section.15	The label of component 1 of the velocities section reads Velocities of Component 1 xyz; the reader reports the declared section as missing.
`v31_label_velocities_0.convel`	`IncompleteSection`, line 16; class-open	CON-section.15	The label of component 1 of the velocities section reads Velocities of Component 0; the reader reports the declared section as missing.
`v31_label_velocities_2.convel`	`IncompleteSection`, line 16; class-open	CON-section.15	The label of component 1 of the velocities section reads Velocities of Component 2; the reader reports the declared section as missing.
`v31_label_velocities_01.convel`	`IncompleteSection`, line 16; class-open	CON-section.15	The label of component 1 of the velocities section reads Velocities of Component 01; the reader reports the declared section as missing.

File	Outcome; basis	Clauses	Reading
`v31_coord_label_mismatch.con`	`ValidationError`, line 11; open	CON-validate.4	The coordinate label reads Coords of Component 1 without validate. CON-validate.4 lists the coordinate label check under validate true, and no clause asks for it in other modes; the reader checks it in every mode.
`v31_label_coordinates_2.con`	`ValidationError`, line 11; class-open	CON-count.8	The coordinate label of component 1 reads Coordinates of Component 2, and component numbers start at 1.
`v31_label_coordinates_1_xyz.con`	`ValidationError`, line 11; open	CON-validate.4	The coordinate label of component 1 reads Coordinates of Component 1 xyz without validate; as for `v31_coord_label_mismatch.con`, the reader checks the label in every mode.
`v31_sections_out_of_order.convel`	`IncompleteSection`, line 16; open	CON-section.1, CON-section.15	sections lists forces then velocities and the blocks are written velocities then forces. No clause fixes the order of the section blocks; the reader reads them in sections order, so the first label fails.
`v31_blank_symbol_velocities.convel`	accept, 1 frame; open	CON-section.4, CON-validate.4	The symbol line of the velocities block is blank. CON-validate.4 checks section symbols under validate true, and no clause says what a symbol line holds in other modes; the reader does not check it.
`v31_section_row_id_mismatch.convel`	accept, 1 frame; optional	CON-section.17	A velocity row's id differs from its coordinate row and validate is absent; the reader may ignore the mismatch, and it does.

File	Outcome; basis	Clauses	Reading
`f26_cell_10_10_170.con`	`ValidationError`, line 4; class-open	CON-cell.6	Round-2 report B, item F26: validate true, lengths 10 10 10 and angles 10 10 170; the radicand of the c vector of CON-cell.6 is negative, so no real cell exists; no clause names the class, and the reader rejects the angles line as ValidationError in every mode.
`gram_170_v1.con`	`ValidationError`, line 4; class-open	CON-cell.6	Angles 170 170 170 in a version 1 frame: the radicand of the c vector of CON-cell.6 is negative, so no real cell exists; no clause names the class, and the reader rejects the angles line as ValidationError in every mode.
`gram_170_v2.con`	`ValidationError`, line 4; class-open	CON-cell.6	Angles 170 170 170 in a version 2 frame: the radicand of the c vector of CON-cell.6 is negative, so no real cell exists; no clause names the class, and the reader rejects the angles line as ValidationError in every mode.
`gram_170_v30.con`	`ValidationError`, line 4; class-open	CON-cell.6	Angles 170 170 170 in a version 3.0 frame: the radicand of the c vector of CON-cell.6 is negative, so no real cell exists; no clause names the class, and the reader rejects the angles line as ValidationError in every mode.

File	Outcome; basis	Clauses	Reading
`gram_170_v31_validate.con`	`ValidationError`, line 4; class-open	CON-cell.6	Angles 170 170 170 in a minor 1, validate true frame: the radicand of the c vector of CON-cell.6 is negative, so no real cell exists; no clause names the class, and the reader rejects the angles line as ValidationError in every mode.
`gram_60_60_120_1.con`	`ValidationError`, line 4; class-open	CON-cell.6	Angles 60 60 120.1, validate absent: the radicand of the c vector of CON-cell.6 is negative, so no real cell exists; no clause names the class, and the reader rejects the angles line as ValidationError in every mode.
`gram_60_60_120_1_validate.con`	`ValidationError`, line 4; class-open	CON-cell.6	Angles 60 60 120.1, validate true: the radicand of the c vector of CON-cell.6 is negative, so no real cell exists; no clause names the class, and the reader rejects the angles line as ValidationError in every mode.
`v31_version_3_5.con`	`InvalidMetadataJson`, line 2; open	CON-line2.4, CON-line2.6	con_spec_version 3.5 is a JSON number that names no integer major. CON-line2.6 makes an unknown major UnsupportedVersion, and the error table makes a reserved key of the wrong type InvalidMetadataJson; the reader takes a major to be an integer and reports InvalidMetadataJson.

File	Outcome; basis	Clauses	Reading
`v2_json_depth_127.con`	accept, 1 frame; open	CON-line2.1	Line 2 nests 127 deep counting the outer object. No clause bounds the nesting depth of line 2, and RFC 8259 section 9 lets a parser set one. The reader accepts 127 levels counting the outer object and rejects deeper nesting.
`v2_json_depth_128.con`	`InvalidMetadataJson`, line 2; open	CON-line2.1, CON-line2.2	Line 2 nests 128 deep counting the outer object. No clause bounds the nesting depth of line 2, and RFC 8259 section 9 lets a parser set one. The reader accepts 127 levels counting the outer object and rejects deeper nesting.
`d06_lattice_1e999.con`	`InvalidMetadataJson`, line 2; open	CON-cell.5, CON-line2.13	Probe D6: 1e999 is a JSON number whose binary64 value is infinite. CON-cell.5 asks for finite numbers, and the error table makes an Inf value NonFiniteValue. A reserved key must hold the type this document gives it (CON-line2.13), and the reader reports a reserved key with a number that is not a finite binary64 as InvalidMetadataJson on line 2.
`d10_json_depth_129.con`	`InvalidMetadataJson`, line 2; open	CON-line2.1, CON-line2.2	Probe D10: line 2 nests 129 deep counting the outer object. No clause bounds the nesting depth of line 2, and RFC 8259 section 9 lets a parser set one. The reader accepts 127 levels counting the outer object and rejects deeper nesting.

File	Outcome; basis	Clauses	Reading
`d11_validate_pbc_object.con`	`InvalidMetadataJson`, line 2; open	CON-cell.4, CON-validate.4	Probe D11: validate true and pbc is an object. The error table makes a reserved key of the wrong type InvalidMetadataJson in every mode, and CON-validate.4 names ValidationError for a bad reserved-key type under validate true. The reader applies the check of every mode first.
`d12_length_trailing_space.con`	accept, 1 frame; open	CON-units.1, CON-units.16	Probe D12: units.length is angstrom with a trailing space. The text does not say whether white space may stand inside a unit expression; the reader removes it.
`d13_blank_symbol_energies.con`	accept, 1 frame; open	CON-section.4, CON-validate.4	Probe D13: the symbol line of the energies block is blank. As for `v31_blank_symbol_velocities.convel`, the reader does not check section symbols outside validate true.
`d15_json_depth_100001.con`	`InvalidMetadataJson`, line 2; open	CON-line2.1, CON-line2.2	Probe D15: line 2 nests 100001 deep, and the reader rejects it without exhausting its stack. No clause bounds the nesting depth of line 2, and RFC 8259 section 9 lets a parser set one. The reader accepts 127 levels counting the outer object and rejects deeper nesting.
`v31_type_count_2pow64.con`	`InvalidCount`, line 7; open	CON-error.3	The type count is 2^64. No clause bounds a count; the reader reads counts as integers in 0 through 2^64-1 and reports a larger one as InvalidCount.

File	Outcome; basis	Clauses	Reading
`v31_gzip_by_content.con`	accept, 1 frame; optional	CON-compress.3	Gzip bytes under a plain .con name; readers should detect gzip by its magic bytes, and this one does.
`v31_bom.con`	`ValidationError`, line 1; class-open	CON-file.2	Begins with the UTF-8 byte-order mark EF BB BF; the clause names no class.
`v31_utf8_line1.con`	`ValidationError`, line 1; class-open	CON-file.1	Line 1 holds the bytes C0 AF, which are not UTF-8; the clause names no class.
`v31_utf8_number_token.con`	`InvalidNumberFormat`, line 12; class-open	CON-file.1, CON-num.8	The byte C0 inside the first coordinate of line 12 is not UTF-8, and the token is not a decimal. CON-num.8 names InvalidNumberFormat and CON-file.1 names no class; the reader reports InvalidNumberFormat.
`v31_utf8_separator_line.convel`	`ValidationError`, line 14; class-open	CON-file.1, CON-section.2	Line 14, where the velocities section needs its blank line, holds the bytes C0 AF, which are not UTF-8; neither clause names a class.

File	Outcome; basis	Clauses	Reading
`v31_unicode_tail.con`	`IncompleteHeader`, line 16; open	CONframe.1, CONframe.3	Lines of U+00A0 U+3000 and of U+2003 follow the last frame. The reader takes a line of Unicode White_Space characters to be blank before a section or a legacy velocity block. The ignored end of the file is one such line, then lines of spaces, tabs, form feeds and carriage returns only. The text does not define whitespace or blank otherwise. The second line holds U+2003, so the two lines are lines 1 and 2 of a further frame, which ends before its line 3.
`ff_tail_610_frames.con`	accept, 610 frames; open	CONframe.1	610 legal frames and a last line holding a form feed. The reader takes a line of Unicode White_Space characters to be blank before a section or a legacy velocity block. The ignored end of the file is one such line, then lines of spaces, tabs, form feeds and carriage returns only. The text does not define whitespace or blank otherwise. So the form feed line is the ignored end of the file.

File	Outcome; basis	Clauses	Reading
`whitespace_only.con`	`InvalidNumberFormat`, line 3; open	CON-frame.1, CON-num.8	Lines 1 and 2 hold a space and a tab, line 3 holds U+3000. The reader takes a line of Unicode White_Space characters to be blank before a section or a legacy velocity block. The ignored end of the file is one such line, then lines of spaces, tabs, form feeds and carriage returns only. The text does not define whitespace or blank otherwise. Line 3 holds U+3000, so the file holds a frame whose line 3 is not a decimal.
`v31_zero_atom_unicode_blank.con`	`InvalidCount`, line 21; open	CON-count.1	A zero-atom frame whose lines 8 and 9 hold U+3000 and U+00A0. The reader takes white space inside a line to be spaces and tabs, the separators of CON-token.3 and the trailing characters of CON-file.4; the text does not define whitespace or blank otherwise. So line 8 holds a token, which CON-count.1 makes InvalidCount.
`v31_label_case_coordinates.con`	accept, 1 frame; open	CON-file.5, CON-validate.4	The coordinate label has leading spaces, a tab between words and upper case. CON-file.5 collapses runs of spaces; the reader compares labels word by word, splitting at spaces and tabs, with ASCII letters folded.
`empty.con`	accept, 0 frames; open	CON-frame.1	Zero bytes. No clause speaks to a file without a frame; the reader returns zero frames.

Table S11 applies the version rule of Eq. (6) and the velocity rule of Section 3 to seven headers of one velocity frame (`paper/cpc/data/failure/version_safety.csv`). The Python module and the C library follow the rule on each row. They reject a velocity block of version 3.1 without a time unit, an unknown major, a missing version and a mask of 9, keep an unknown key of a later minor, and read a CON 3.0 block labelled fs in the natural unit. The Python rewrite of that block stays at version 3.0 and keeps its fs label, so the next reader again applies the natural unit.

Table S9: Both corpora on the frame readers and the frame count of the Python module and the C library. A cell gives the reads that return the outcome of the manifest, of all reads at that thread setting. The 165 files are the 22 of the conformance corpus and the 143 of the CON 3.1 corpus. A read matches when it gives the manifest's outcome and frame count, or its error class; neither binding reports a line, and where the manifest names no frame count every read of the file gives the same one. The plain file is the corpus file; the mapped file is the decoded text at the mapping threshold or beyond, with legal 13-line frames in front of a smaller text, or after a text rejected on its first line, which add their frames to the count; the gzip file is the decoded text compressed with gzip. A Python str cannot hold the 3 files that are not UTF-8, so `read_con_string` reads the others. The C library has no zstd in either build and rejects the 2 zstd files in the plain file, file iterator and frame count rows with the message that names the feature, as it documents. The default build has no Rayon and parses on one thread at every setting, and the release archives contain the parallel build.

Build	Entry point	1 thread	2 threads	8 threads	Default	Serial
Python module	`read_con_string`	162/162	162/162	162/162	162/162	–
	`read_con`, plain file	165/165	165/165	165/165	165/165	–
	`read_con`, mapped file	165/165	165/165	165/165	165/165	–
	`read_con`, gzip file	165/165	165/165	165/165	165/165	–
	`iter_con`, plain file	–	–	–	–	165/165
	`iter_con(mmap=True)`, mapped file	–	–	–	–	165/165
	`iter_con(stream=True)`, plain file	–	–	–	–	165/165
	`iter_con(stream=True)`, gzip file	–	–	–	–	165/165
	`count_frames`, plain file	–	–	–	–	165/165
	`count_frames`, mapped file	–	–	–	–	165/165
	`count_frames`, gzip file	–	–	–	–	165/165
C library, default build	`read_con_buffer_iterator`	–	–	–	–	165/165
	`rkr_read_all_frames_n_threads`, plain file	165/165	165/165	165/165	165/165	–
	`rkr_read_all_frames_n_threads`, mapped file	165/165	165/165	165/165	165/165	–
	`rkr_read_all_frames_n_threads`, gzip file	165/165	165/165	165/165	165/165	–
	`read_con_file_iterator`, plain file	–	–	–	–	165/165
	`rkr_count_frames`, plain file	–	–	–	–	165/165
	`rkr_count_frames`, mapped file	–	–	–	–	165/165
	`rkr_count_frames`, gzip file	–	–	–	–	165/165
C library with parallel	`read_con_buffer_iterator`	–	–	–	–	165/165
	`rkr_read_all_frames_n_threads`, plain file	165/165	165/165	165/165	165/165	–
	`rkr_read_all_frames_n_threads`, mapped file	165/165	165/165	165/165	165/165	–
	`rkr_read_all_frames_n_threads`, gzip file	165/165	165/165	165/165	165/165	–
	`read_con_file_iterator`, plain file	–	–	–	–	165/165
	`rkr_count_frames`, plain file	–	–	–	–	165/165
	`rkr_count_frames`, mapped file	–	–	–	–	165/165
	`rkr_count_frames`, gzip file	–	–	–	–	165/165

Table S10: readcon-core and the independent reader over the conformance corpus and the CON 3.1 corpus. Each entry of the CON 3.1 corpus manifest names acceptance with a frame count, or rejection with an error class and a line, and the basis of that outcome in the released CON 3.1 text (`docs/orgmode/spec.org`, 125 clauses): explicit when clauses decide the outcome and the class, class-open when clauses require the rejection and name no class, optional when a clause permits the outcome without requiring it, and open when no clause decides the file. The corpus has 95 explicit, 21 class-open, 4 optional and 23 open entries. The conformance manifest names acceptance, or rejection with an error class, and its entries count as explicit. A reader matches an entry when it returns every value that the entry names. These are the verdict, the frame count, the error class, the line of the error where the reader reports one, and the periodicity and the column-4 mask of the first frame where the entry names them. The reader also warns of a time-unit override exactly on the entries that name one. For `read_con`, the rewrite of each file that its entry marks keeps the version, minor and units of every frame of major 3. It writes a frame of major 1 or 2 as major 3, with each unit that the frame declares and the same fields, as the Python, Julia and Fortran writers do. That rewrite keeps the minor of a frame that declares one or whose velocities carry a time label, which pins it to minor 0; a frame of version 3.0 with neither comes back at minor 1, and no marked file contains one. A reader that differs from an explicit entry disagrees with the specification, and one that differs from any other entry departs from the reading that entry records. readcon-core 0.18.1 puts no line number in its Python exceptions, so its lines are not compared; it warns of a declared time unit that it does not apply only when it converts a frame to ASE (`ConFrame.to_ase`), and the check converts every frame that has velocities. `read_con` and `iter_con` are two paths through readcon-core; the independent reader is pure Python written from the released text, and it follows the reading the manifest records where that text leaves a case open. Field-identical counts the files that both `read_con` and the independent reader accept with equal fields: coordinates, velocities and cell equal in every bit, and the same masks, identifiers, symbols, masses, version, minor, declared and applied time unit and periodicity.

		`read_con`			`iter_con`			Independent			
File group	Files	Match	Disagree	Not followed	Match	Disagree	Not followed	Match	Disagree	Not followed	Field-identical
Conformance, valid	14	14	0	0	14	0	0	14	0	0	14/14
Conformance, invalid	8	8	0	0	8	0	0	8	0	0	–
CON 3.1 corpus, accept	62	62	0	0	62	0	0	62	0	0	62/62
CON 3.1 corpus, reject	81	81	0	0	81	0	0	81	0	0	–
All	165	165	0	0	165	0	0	165	0	0	76/76

Table S11: Header variants of one velocity frame. Rule is what the specification requires. Where the rule names a time unit, the Python cell gives the time unit of the Python rewrite.

Variant	Rule	Python	C
3.1, natural unit declared	accept	accept	accept
3.1, no time unit	reject	reject	reject
3.0, labelled fs	accept (natural)	accept, fs	accept
3.3, unknown key	accept, keep	accept, kept	accept
4.0	reject	reject	reject
missing version	reject	reject	reject
3.1, mask 9	reject	reject	reject

S5. Survey frame, grading and drift files

The survey frame of Table S3 is a Pt(100) slab of $2 \times 2 \times 2$ atoms with 6 Å of vacuum and one Pt adatom 1.8 Å above the hollow site; Listing 1 prints it. Every coordinate carries a uniform offset of up to 10^{-3} Å from a fixed seed, so writing it back takes all seventeen significant digits. The bottom four atoms are held in every component, and the adatom is held in z. The identifiers are 17, 3, 11, 5, 23, 2, 13, 7 and 41 in row order. Velocities are normal deviates of standard deviation 4×10^{-3} in the natural unit, zero on held components.

Each library is given each field through the mechanism its own documentation names, Table S12. A field counts as kept when it reads back equal. A velocity is declared when the file names its unit and the values agree after conversion to within 10^{-6} relative. A velocity has "no unit" when it comes back from a file that names no time unit, whatever unit the reader then applies. Read as Å/fs, the natural-unit velocities of the frame give a kinetic energy 103.64 times too large. One rule labels a field that came back. It is fixed by the format when the format's own specification defines the name and meaning of the place it was stored in, and a convention when only a library's documented layout defines that place, so only readers that share the layout get it back. A label is an identifier that came back only inside the site name of a Crystallographic Information File (CIF). A file is named "H5MD, private mask dataset" when it has the H5MD root group and its version and keeps the mask in a dataset that H5MD does not define.

Each drift file of Table S2 has three Pt atoms with the header

```
Lattice="5.5437171645025325 0.0 0.0 0.0 5.5437171645025325 0.0 0.0 0.0
    20.0" Properties=species:S:1:pos:R:3:move_mask:L:3:id:I:1 pbc="T T F"
```

Table S12: What each library is given for the mask, the identifiers, the velocities and the periodicity of the survey frame.

Library and file	Mechanism
ASE, extended XYZ and POSCAR	One FixAtoms for the bottom atoms, and for the adatom one FixCartesian on z in extended XYZ and one FixScaled on z in POSCAR; the identifiers as the integer array `id`; velocities as ASE momenta; `pbc`.
ASE trajectory	The constraints of extended XYZ, the identifiers as ASE tags, momenta and `pbc`.
chemfiles	The `id` and `move_mask_0` to `move_mask_2` atom properties and a `pbc` frame property.
OVITO	The Particle Identifier and Velocity properties and an integer `move_mask` property, written at seventeen significant digits.
h5py, H5MD 1.1	The standard `id`, `position`, `velocity` with its unit and `box` elements, and the mask in a private `fixed_mask` dataset.
MDAnalysis, H5MD	Names, identifiers, positions, the box, the units, and velocities converted to Å/ps, its base unit.
Open Babel, CIF and XYZ	The identifier in `_atom_site_label` and a P 1 cell.

and rows `Pt 0.0 0.0 6.0 T T T 17`, `Pt 2.7718585822512662 0.0 6.0 T T T 3` and `Pt 1.3859292911256331 1.3859292911256331 7.8 F F T 41`. The variants drop the `pbc` key, replace the header by a plain comment and the rows by their first four fields, print the z value 6.0 as `6.0d0`, spell the booleans `True` and `False`, add `note="E=1.5 eV"`, and add `units="angstrom,eV,fs"`. The readers are ASE 3.29.0, the libAtoms `extxyz` 0.4.6 package [5] through its C path and its Python path, chemfiles 0.10.4, the Open Visualization Tool (OVITO) 3.16.1, and the plain Cartesian coordinate (XYZ) readers of MDAnalysis 2.10.0 and Open Babel 3.2.1. On the reference file chemfiles moves 3 coordinates, the x of row 2 and the x and y of row 3, by 1, 2 and 2 units in the last place from the binary64 values nearest their decimals (`paper/cpc/data/failure/drift_extxyz.json`). The binary64 value nearest 2.7718585822512662 has the bit pattern `40062cc4313f3fb9`; chemfiles returns `40062cc4313f3fb8`.

S6. Records beside the checkpoint

Table S13 answers four questions for the checkpoint and for the other routes of Table S3. Restart-exact means binary64 coordinates, the per-component mask, the identifier and the periodic flags all come back. Velocities is how the file states the velocity unit. Metadata is the version or schema that another reader can apply. Indexed means frame k can be opened without a scan whose length is the number of earlier frames. Every row is a route measured in Table S3. Tables S14 and S15 give, one row per format, the version, the atom identifiers, the per-component mask, the velocity time unit and the decimal round trip as the specification of each format of Table S13 and of other trajectory, structure and quantum-chemistry formats states them, with the source of every cell. The trajectory next generation (TNG) format and the trajectories of GROMACS and CHARMM [6, 7, 8, 9], and the NetCDF trajectory of Amber [10, 11, 12], are not routes of Table S3. Other records sit beside a saddle search without being measured here: the archives of the Protein Data Bank (PDB), wwPDB and mmCIF [13, 14, 15]; the column files of LAMMPS and the Atomsk converter [16, 17]; the quantum-chemistry objects of cclib, IOData, TREXIO and QCArchive [18, 19, 20, 21]; the published geometries of OPTIMADE, the Novel Materials Discovery (NOMAD) laboratory, Catalysis-Hub, Transition1x and OC20 [22, 23, 24, 25, 26]; the analysis tools MDTraj, Visual Molecular Dynamics (VMD), d-SEAMS, essential dynamics and parallel H5MD analysis over the Message

Table S13: Restart record against the other routes of Table S3. "Scan" means the reader walks the file to find a frame. "Offsets" means the reader records a byte offset during that walk, which stays serial because each start depends on the counts before it.

Record	Restart-exact	Velocities	Metadata	Indexed
CON 3.1	yes	declared	versioned	offsets
extended XYZ, ASE writer	no, 8 decimals	no unit	comment	scan
extended XYZ, OVITO writer	no, periodicity	no unit	comment	scan
ASE trajectory, identifiers as tags	yes, by convention	no unit	ULM and ASE versions	yes
POSCAR, FixScaled	no, mask only	no unit	none	no
H5MD (h5py), private mask dataset	yes, mask by convention	declared	versioned	yes
H5MD, MDAnalysis	single precision	declared	versioned	yes
CIF, Open Babel	no	omitted	none	no

Passing Interface (MPI) [27, 28, 29, 30, 31]; the array stores Zarr and ADIOS2 and portable scientific data [32, 33, 34]; the initial boxes of PACKMOL [35]; the program checkpoints of PLUMED and ESPResSo [36, 37]; and the live steps of i-PI and the MolSSI Driver Interface (MDI) [38, 39, 40, 41].

Table S14: The version, the atom ids and the per-component mask in the specification of each format, one row per format. A cell restates the source of its row's letter unless it carries its own letter. The list below the table gives the sources. Table S3 measures round trips through those of these formats that its libraries write.

Format	Version in the file	Atom ids	Per-component mask
H5MD 1.1.0-rc1[a]	yes; major and minor stored as Integer[2]	yes	not among the standardized particle identifiers
extended XYZ[c]	no in-frame version field in the specification section	not a reserved column	not stated in the specification section
ASE trajectory[d]	yes; version=1 plus ase_version	not stated; optional tags have no reorder rule	constraints stored as encoded dicts; no mask-bit field
TNG 1.8.2[e]	yes; a 64 bit version in every block header	yes; particle mapping blocks give the real particle number of each stored particle	not among the standard blocks
TRR[g]	not stated in the trr section	not stated in the trr section	not stated in the trr section
XTC[g]	yes; magic 1995 for the current file version	not in the field list	not in the field list
AMBER NetCDF 1.0 revision C[h]	yes; ConventionVersion must be 1.0	no; atom is an index dimension	not stated in the variables section
DCD[i]	plugin writes Charmm 24; no spec-version field in the header layout	not in the layout	no; a fixed-atom count only
LAMMPS text dump[j]	manual is per LAMMPS release; no in-file version token stated for the text snapshot	yes when the id column is written	not in the attribute list
POSCAR[k]	no in-file version token stated	not stated; order follows the species counts	yes; selective dynamics
CIF 1.1 and PDBx mmCIF[l]	yes for the syntax comment; dictionary version on the item pages	yes as a unique record id[m]	not stated in the syntax page or the four dictionary items fetched
PDB format 3.3[o]	the document is version 3.3; no in-file spec version on these pages	serial numbers a record and is not defined as stable across models	not stated; occupancy is not a constraint mask
TREXIO[p]	library version in the file; no separate format-spec version attribute	labels and indices; no reorder rule stated	not stated; nuclei are fixed point charges
CON 3.1[q]	yes; majors 1 2 3.0 and 3.1 with minor rules	yes; column 5	yes; bits 0 1 and 2 from spec 2

Sources: a https://www.nongnu.org/h5md/h5md.html;
c https://github.com/libAtoms/extxyz/blob/master/README.md; d ASE 3.29.0 ase/io/trajectory.py;
e TNG 14dd68f Trajectoryformatspecification.mk; g GROMACS 2026.4 Reference Manual File formats;
h https://ambermd.org/netcdf/nctraj.xhtml; i fetched page titled DCD Plugin Version 1.11 on www.ks.uiuc.edu/Research/vmd/; j https://docs.lammps.org/dump.html;
k https://www.vasp.at/wiki/index.php?title=POSCAR&oldid=37632;
l https://www.iucr.org/resources/cif/spec/version1.1/cifsyntax;
m https://mmcif.wwpdb.org/dictionaries/mmcif_pdbx_v50.dic/Items/_atom_site.id.html;
o https://www.wwpdb.org/documentation/file-format-content/format33/sect9.html;
p https://trex-coe.github.io/trexio/trex.html; q docs/orgmode/spec.org.

Table S15: The velocity time unit and the decimal round trip of the coordinates in the specification of each format, one row per format (the formats of Table S14). A cell restates the source of its row's letter unless it carries its own letter. The list below the table gives the sources.

Format	Velocity time unit	Decimal round trip
H5MD 1.1.0-rc1[a]	yes through the units module; an optional unit attribute on the value and time datasets[b]	binary Float or Integer; no decimal round-trip rule
extended XYZ[c]	no; velo is mapped to momenta	not stated
ASE trajectory[d]	no; momenta with no time unit	binary arrays; no decimal round-trip rule in this module
TNG 1.8.2[e]	no unit field for velocities; the library stores a distance-unit exponent and frame times in seconds[f]	no decimal form; binary32 or binary64 uncompressed, integers times a multiplier when compressed
TRR[g]	velocities are stored; no time-unit label stated	binary full precision in the index; no decimal rule
XTC[g]	no velocities; time is a float with no unit name	no
AMBER NetCDF 1.0 revision C[h]	yes; angstrom/picosecond	binary float or double; no decimal rule
DCD[i]	not in the layout shown	DOUBLE coordinates in the layout; no decimal rule
LAMMPS text dump[j]	velocities as vx vy vz; no time-unit label in the text snapshot	no
POSCAR[k]	velocities optional; no time-unit label in the block	no; at least 7 significant digits recommended
CIF 1.1 and PDBx mmCIF[l]	not stated in the items consulted[n]	not stated
PDB format 3.3[o]	not stated in the coordinate section	no; coordinates are Real(8.3)
TREXIO[p]	not stated in the nucleus or metadata groups	not stated; values are typed float
CON 3.1[q]	yes from minor 1	yes for the default printer; fixed decimals are opt-in and need not round-trip

Sources: a https://www.nongnu.org/h5md/h5md.html;
b https://www.nongnu.org/h5md/modules/units.html;
c https://github.com/libAtoms/extxyz/blob/master/README.md; d ASE 3.29.0 ase/io/trajectory.py;
e TNG 14dd68f Trajectoryformatspecification.mk; f TNG 14dd68f src/lib/tng_io.c; g GROMACS 2026.4 Reference Manual File formats; h https://ambermd.org/netcdf/nctraj.xhtml; i fetched page titled DCD Plugin Version 1.11 on www.ks.uiuc.edu/Research/vmd/; j https://docs.lammps.org/dump.html;
k https://www.vasp.at/wiki/index.php?title=POSCAR&oldid=37632;
l https://www.iucr.org/resources/cif/spec/version1.1/cifsyntax;
n https://mmcif.wwpdb.org/dictionaries/mmcif_pdbx_v50.dic/Items/_atom_site.Cartn_x.html;
o https://www.wwpdb.org/documentation/file-format-content/format33/sect9.html;
p https://trex-coe.github.io/trexio/trex.html; q docs/orgmode/spec.org.

S7. Restart protocol

The restart study of Table 7 uses a Pt(100) slab of $3 \times 3 \times 3$ atoms under the effective-medium theory (EMT) potential of ASE, with 6.0 Å of vacuum, the bottom layer held, and one adatom 1.75 Å above a hollow and held in z, hopping to the neighbouring hollow. Endpoints were relaxed with the Broyden--Fletcher--Goldfarb--Shanno (BFGS) optimiser of ASE to 0.002 eV/Å. Five movable images were interpolated with the constraints applied. The band uses the improved tangent with a climbing image, and each step adds 0.02 Å^2/eV times the projected force. The band is interrupted after step 25. Each route writes all seven images and reads them back with the reader the route names, keeping whatever constraints and periodicity that reader returns, and the resumed band runs to 0.01 eV/Å or the cap of 4,000 steps.

The counts in units in the last place of Table 7 were recorded on host `Intel(R) Xeon(R) Processor`, family 6, model 207 with the kernels of `scripts/numeric_pins.py`: one OpenBLAS thread on its Nehalem kernels and NumPy at its x86-64-v2 baseline, which every x86-64-v2 host runs. File, $\Delta E^{\ddagger}$ and the displacement are printed to the significant figures, three at most, on which reruns of every route on four kernel sets of `scripts/numeric_pins.py` on that host agree, and as a range where their first figures differ: OpenBLAS Nehalem with NumPy at x86-64-v2, the kernels of the record, OpenBLAS Cooperlake with NumPy at x86-64-v4 with `AVX512_ICL` and `AVX512_SPR`, OpenBLAS Haswell with NumPy at x86-64-v3 and OpenBLAS SkylakeX with NumPy at x86-64-v4, the kernels that hosts of those generations select when nothing is pinned. Every rerun takes the steps of the table, the wall times are those of host `Intel(R) Xeon(R) Processor`, x86_64, 8 processing units, and the build is readcon 0.18.1 with ASE 3.29.0.

S8. Velocity unit and identifiers

Figure S1 is the step of Section 7 on the 28-atom fixture. The fixture declares the natural unit, 10.180505711 fs for the CODATA 2018 constants. A misread scales the kinetic energy by the square of that number, 103.64. The step integrates the EMT forces of the fixture with velocity Verlet over 1 fs, once with the declared unit and once with the numbers read as Å/fs. Each bar is the length of that displacement for one atom.

Frames 0 and 1 of one file are the public endpoints [1] of hydrogen cyanide (HCN) and hydrogen isocyanide (HNC), the two minima of the HCN--HNC

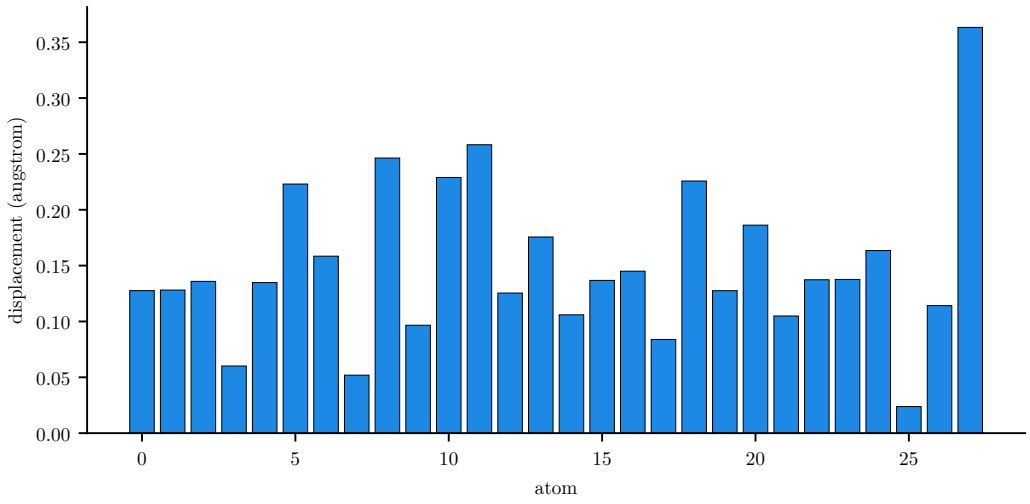


Figure S1: Position difference after one velocity Verlet step of 1 fs on the 28-atom fixture, between the declared natural unit and the same numbers read as Å/fs. The kinetic energy goes from 92.02 eV to 9,537 eV, a ratio of 103.64. The root mean square over the Cartesian components is 0.096 Å and the largest component 0.315 Å. Each bar is the length of the displacement of one atom.

system [42]. On both, readcon-core returns every atom free, the identifiers `0 1 2` and the declared natural unit. Frame 2 is a constructed probe with the identifiers `2 0 1` and hydrogen fixed on x alone. Table S16 is what readcon-core returns on each atom of the three frames (`paper/cpc/bench/results/hcn_loss.csv`).

Table S16: Per-atom mask, id and velocity returned by readcon-core. A mask is the integer m of Eq. (3). The probe stores ids 2, 0 and 1 and fixes hydrogen on x only.

Atom	Mask	Id	Velocity
HCN C	0	0	present
HCN N	0	1	present
HCN H	0	2	present
HNC C	0	0	present
HNC N	0	1	present
HNC H	0	2	present
probe C	0	2	present
probe N	0	0	present
probe H	1	1	present

S9. Timing

The candidate timing campaign is not reported. Section 8 of the main text uses the measurements in Section 9. Those logs are `paper/cpc/logs/reader-exchange.log`, `paper/cpc/logs/db.log`, `paper/cpc/logs/mpi.log` and `paper/cpc/logs/dlpack-exchange.log`. Each head-to-head arm reports how many frames match the reference and how many do not. A time is the median of the runs of its cell [43, 44]. The ordering table tests those ratios with the rank sum statistic of Mann and Whitney [45], the interval of Hodges and Lehmann [46] and Holm's step-down rule [47]. memray records the peak heap of a cell [48]. libxtc reads the XTC trajectory [49]. Resumed bands are drawn on the (s, d) plane [2] after iterative rotational alignment [50]. MDTraj is an analysis tool beside these readers [27]. Software citation follows the principles for research software [51]. The tensor exchange sits beside the metatensor layout [52].

Table S17 is context for choosing a format. readcon reads neither XYZ nor HDF5. h5py reads the same frames in 0.5937 s against readcon's 1.564 s on the coordinate file. The mapped binary coordinate file reads those frames in 0.006952 s. The chemfiles row copies each frame out of the trajectory iterator, and that copy matches 100000 of 100000 frames.

Table S17: Context for choosing a format, each call at 1 thread. These readers open XYZ, HDF5, or the mapped binary coordinate file. readcon does not read XYZ or HDF5.

Reader	Median (s)	IQR (s)	MiB	Matched
NumPy `loadtxt`, 1 thread	1.249	0.0295	391	100000/100000
chemfiles positions, 1 thread	1.428	0.0387	166.7	100000/100000
OVITO positions, 1 thread	5.014	0.0918	172.3	100000/100000
ASE extended XYZ `get_positions`, 1 thread	7.866	0.0778	345.8	100000/100000
pymatgen `cart_coords`, 1 thread	24.8	0.571	2119	100000/100000
h5py dataset, 1 thread	0.5963	0.00797	758.7	100000/100000
MDAnalysis positions, 1 thread	34.47	0.414	139.4	100000/100000
CON-bin `read_conbin`, 1 thread	0.007035	0.00337	146.8	100000/100000

S10. Measured versions

The survey of Table S3 ran ASE 3.29.0, chemfiles 0.10.4, MDAnalysis 2.10.0, h5py 3.16.0, OVITO 3.16.1 and Open Babel 3.2.1. chemfiles 0.10.4, MDAnalysis 2.10.0, OVITO 3.16.1 and Open Babel 3.2.1 open neither a version-1 CON file nor a CON 3.1 file, and the eon module of ASE 3.29.0 opens both (`scripts/con_openers.py`). The drift files of Table S2 went to ASE 3.29.0, the libAtoms `extxyz` 0.4.6 package, chemfiles 0.10.4, OVITO 3.16.1, MDAnalysis 2.10.0 and Open Babel 3.2.1. The readers timed on this branch are named in Section 9 of the main text.

`scripts/spec_version_drift.py` gives the probe files to the published readcon wheels of 0.17.0, 0.17.1, 0.17.2 and 0.17.3, each in its own process, and the four do not return the same outcome on every file (`paper/cpc/data/spec/version_drift.json`). On files of two frames whose line 1 is empty, the first refuses the file and the later three read two frames. Only the second refuses a file with a legacy velocity label in capitals or separated by tabs, which the other three read. On a blank symbol line and on a symbol line that starts with `{`, the third and the fourth refuse the file and the earlier two read it, and the fourth returns the outcome of the third on every probe.

S11. Reproduction

Table S18 names the build and host of every record from a readcon-core build. This checkout is readcon-core 0.18.1, the untagged commit `f2cea7e8`. The library of this checkout has a digest that begins `d94b6ad922e9`. This checkout rejects a cell whose radicand lies outside the rounding window of `cell_vectors`, and a declared atom count larger than the bytes that

remain. The reader exchange in `paper/cpc/logs/reader-exchange.log` was measured on the clean commit `a9334b1562581adf25d32d83003ba66daf8e6514`. The stored campaigns in Table S18, including r0174 and the head-to-head records, were not rerun on that commit. Each of those records names the library it was measured with, and that digest begins `d94b6ad922e9`. Two of them also run other builds, which their headings name: the drift comparison reads the published wheels of 0.17.0, 0.17.1, 0.17.2 and 0.17.3, the earlier releases of CON 3.1, and the eOn case runs the public eOn client and the readcon-core it embeds. `scripts/provenance.py` writes the digests of the readcon module and of the library sources into each record. The version history of Table 5, `data/spec/versions.json`, is read from `spec.org` at each release tag by `gen_spec_supp.py --measure`, which runs no readcon-core code, so the table does not list it. Nor does it list `data/spec/clause_coverage.json`, which `clause_coverage.py` reads from `spec.org`, the corpus manifests, the probes and the sources of the tests without running readcon-core.

Table S18: The build and host of every record behind a printed result. Each heading names the readcon-core version, the library digest (the SHA-256 over the library sources, leading hexadecimal digits), the Python module or binary behind the records below it with its SHA-256, the Rust compiler, the Python interpreter and the host, as far as those records name them. Each record names the tables, figures, listings and parts of the documents that print a value from it.

Record	Result
readcon-core 0.18.1, library `d94b6ad922e9`; module `site-packages/readcon/readcon.cpython-312-x86_64-linux-gnu.so`, SHA-256 `5039fd2741b8`; rustc 1.88.0, Python 3.12.16; family 6 model 207, 2400 MHz, 8 threads	
`bench/results/r0174/`	Front matter, Figure 2, Figure 3, Figure 4, Figure 5, Figure 6, Figure 7, Figure 8, Figure 10, Figure 11, Figure 12, Figure 13, Figure 14, Figure 15, Figure 16, Figure 17, Figure 18, Figure 19, Figure 20, Figure 21, Figure 22, Figure 23, Figure 24, Figure 25, Figure 26, Figure S1, Generated records, Table S24
`bench/results/velocity_case.json`	Front matter, Section 1, Section 2, Figure 2, Figure 3, Figure 4, Figure 5, Section 7, Figure 6, Figure 7, Figure 8, Figure 10, Figure 11, Figure 12, Figure 13, Figure 14, Figure 15, Figure 16, Figure 17, Figure 18, Figure 19, Figure 20, Figure 21, Figure 22, Figure 23, Figure 24, Figure 25, Figure 26, Section S1, Section S5, Section S8, Figure S1, Generated records
`data/corpus_crosscheck.json`	Abstract, Section 6, Section 11, Table S10
`data/failure/bands/`	Front matter, Figure 2, Figure 3, Figure 4, Figure 5, Figure 6, Figure 7, Figure 8, Figure 10, Figure 11, Figure 12, Figure 13, Figure 14, Figure 15, Figure 16, Figure 17, Figure 18, Figure 19, Figure 20, Figure 21, Figure 22, Figure 23, Figure 24, Figure 25, Figure 26, Figure S1, Generated records

Record	Result
`data/failure/con_openers.json`	Section S10
`data/failure/drift_extxyz.csv`	Section 6, Table S2, Section S5
`data/failure/drift_extxyz.json`	Section 6, Section S5
`data/failure/matrix.csv`	Front matter, Figure 2, Figure 3, Figure 4, Figure 5, Figure 6, Figure 7, Figure 8, Figure 10, Figure 11, Figure 12, Figure 13, Figure 14, Figure 15, Figure 16, Figure 17, Figure 18, Figure 19, Figure 20, Figure 21, Figure 22, Figure 23, Figure 24, Figure 25, Figure 26, Table S3, Figure S1, Generated records
`data/failure/matrix.json`	Section 7
`data/failure/restart_bands.json`	Table 7, Section S7
`data/failure/restart_formats.csv`	Section 7, Table 7, Section S7
`data/failure/restart_formats.json`	Section 7, Table 6, Table 7, Section 9.2, Section S7
`data/failure/restart_kernels/cooperlake.json`	Table 7, Section S7
`data/failure/restart_kernels/haswell.json`	Table 7, Section S7
`data/failure/restart_kernels/nehalem.json`	Table 7, Section S7
`data/failure/restart_kernels/skylakex.json`	Table 7, Section S7
`data/failure/version_safety.csv`	Table S11
`data/restart_walls/restart_walls.csv`	Table 7, Section S7
`data/restart_walls/restart_walls.json`	Table 7, Section S7
`data/sizes.csv`	Table 9
`data/spec/corpus_recount.json`	Section 6, Table S1
`data/survey_frame.json`	Listing 1
`evidence/results/e5_prefactor.json`	Abstract, Section 7.2, Table 8, Section 11
`src/figures/generated/survey_frame.con`	Listing 1

readcon-core 0.18.1, library `d94b6ad922e9`; module `site-packages/readcon/readcon.cpython-312-x86_64-linux-gnu.so`, SHA-256 `5039fd2741b8`; rustc 1.88.0, Python 3.12.16; family 6 model 207, 2400 MHz, 8 threads; eOn 3.6.0 (`np2py314h465d2a2_0`), which embeds readcon-core 0.16.0

`evidence/results/e1_restart.json`	Section 3, Section 4
`evidence/results/e1_restart_lbfgs_control.json`	Section 3, Section 4
`evidence/work/e1/neb/full/`	Front matter, Figure 2, Figure 3, Figure 4, Figure 5, Figure 6, Figure 7, Figure 8, Figure 10, Figure 11, Figure 12, Figure 13, Figure 14, Figure 15, Figure 16, Figure 17, Figure 18, Figure 19, Figure 20, Figure 21, Figure 22, Figure 23, Figure 24, Figure 25, Figure 26, Figure S1, Generated records
`evidence/work/e1/neb/resume_fixed6/`	Front matter, Figure 2, Figure 3, Figure 4, Figure 5, Figure 6, Figure 7, Figure 8, Figure 10, Figure 11, Figure 12, Figure 13, Figure 14, Figure 15, Figure 16, Figure 17, Figure 18, Figure 19, Figure 20, Figure 21, Figure 22, Figure 23, Figure 24, Figure 25, Figure 26, Figure S1, Generated records
`evidence/work/e1/neb/resume_round/`	Front matter, Figure 2, Figure 3, Figure 4, Figure 5, Figure 6, Figure 7, Figure 8, Figure 10, Figure 11, Figure 12, Figure 13, Figure 14, Figure 15, Figure 16, Figure 17, Figure 18, Figure 19, Figure 20, Figure 21, Figure 22, Figure 23, Figure 24, Figure 25, Figure 26, Figure S1, Generated records

readcon-core 0.18.1, library `d94b6ad922e9`

Record	Result
`data/test_run.json`	Section 2, Section S3, Listing S1
`data/worked_example.json`	Section 1, Table 1, Section 2, Section 3, Section 4, Algorithm 1, Table S5, Section S3
readcon-core 0.18.1, library `d94b6ad922e9`; module `site-packages/readcon/readcon.cpython-312-x86_64-linux-gnu.so`, SHA-256 `5039fd2741b8`; rustc 1.88.0, Python 3.12.16; family 6 model 207, 2400 MHz, 8 threads; binary `paper/cpc/bench/readcon-bench/target/release/readcon-bench`, SHA-256 `c9c31aa59099`	
`bench/results/hcn_loss.csv`	Table S16
readcon-core 0.18.1, library `d94b6ad922e9`; module `site-packages/readcon/readcon.cpython-312-x86_64-linux-gnu.so`, SHA-256 `5039fd2741b8`; rustc 1.88.0, Python 3.12.16; family 6 model 207, 2400 MHz, 8 threads; commit `82214148729bf7b77922c7e7a148925b013c2cac`	
`data/spec/mutant_differential.json`	Section 6, Section S4
readcon-core 0.18.1, library `d94b6ad922e9`; module `site-packages/readcon/readcon.cpython-312-x86_64-linux-gnu.so`, SHA-256 `5039fd2741b8`; rustc 1.88.0, Python 3.12.16; family 6 model 207, 2400 MHz, 8 threads; commit `82214148729bf7b77922c7e7a148925b013c2cac`; C libraries `target/release/libreadcon_core.so`, SHA-256 `727a723102fd` and `target/clib-parallel/release/libreadcon_core.so`, SHA-256 `f781f8ddd50c`	
`data/corpus_paths.json`	Section 6, Section S4, Table S9
readcon-core 0.18.1, library `d94b6ad922e9`; module `site-packages/readcon/readcon.cpython-312-x86_64-linux-gnu.so`, SHA-256 `5039fd2741b8`; rustc 1.88.0, Python 3.12.16; family 6 model 207, 2400 MHz, 8 threads; measures the published readcon-core 0.17.0, 0.17.1, 0.17.2, 0.17.3	
`data/spec/version_drift.json`	Section 6, Section S10, Section S11
readcon-core 0.18.1, library `d94b6ad922e9`; module `target/release/examples/corpus31_report`, SHA-256 `638ca2dfc402`; rustc 1.88.0; family 6 model 207, 2400 MHz, 8 threads; commit `82214148729bf7b77922c7e7a148925b013c2cac`	
`data/corpus31-results.json`	Section 6, Section S4, Table S8
readcon-core 0.18.1, library `d94b6ad922e9`; rustc 1.88.0; family 6 model 207, 2400 MHz, 8 threads	
`data/every_path.json`	Section S4

The git bundle `readcon-core.bundle` of the supplementary material is a complete history, so `git clone readcon-core.bundle` needs no other commit. Its HEAD contains `paper/cpc`. `paper-cpc.tar.gz` is that directory packed beside the bundle, with the scripts, the records and the TeX sources of both documents. Listing S2 clones that commit from the bundle without a network, unpacks `paper/cpc` into the checkout, installs the Python environment that ran the scripts, and rebuilds both documents. It needs git, conda and TeX Live or Tectonic. `scripts/build_tex.sh` typesets both documents with pdfTeX and BibTeX, and on `Ubuntu 24.04.5 LTS` it runs the TeX Live of the distribution, whose `pdftex --version` prints `pdfTeX 3.141592653-2.6-1.40.25 (TeX Live 2023/Debian)`. Apart from the TeX sources of the two documents and the configuration, font map and format that the installation generates, every file that pdfTeX and BibTeX read for them belongs to one of nine Ubuntu packages. Seven of them have version `2023.20240207-1`: `texlive-base`, `texlive-latex-`

base, texlive-latex-recommended, texlive-latex-extra, texlive-science, texlive-publishers and texlive-plain-generic. The other two are texlive-binaries, at version 2023.20230311.66589-9build3, and lmodern, at version 2.005-1. apt install with the arguments texlive-latex-extra, texlive-science, texlive-publishers, texlive-plain-generic and lmodern installs the nine with the packages they depend on. On a host without TeX Live, scripts/build_tex_tectonic.sh typesets both documents with Tectonic 0.17.0 on the path, three rounds of each so that the labels of each document reach the other, and stops on an undefined reference, an overfull box or a float that fits on no page. paper/cpc/environment/conda-linux-64.txt lists the conda-forge packages of that environment with the MD5 of each, among them Python 3.12, pandoc 3.13 and the rustc of the checkout, and requirements.txt beside it the packages that pip adds; scripts/export_env.py wrote both from the package records of the environment. pip install . builds the readcon module from the checkout. scripts/regen_paper.py rewrites every table, figure and macro from the records without timing anything, and scripts/build_tex.sh or scripts/build_tex_tectonic.sh typesets both documents. scripts/trace_numbers.py then traces every printed number to its record and writes paper/cpc/data/trace/numbers.csv.

The measurements are separate steps, each of which rewrites its own records. regen_paper.py --measure reruns the survey, the survey listing, the worked example of Listing 1, the drift files, the toolkits that open CON, the version-safety check, the corpus recount, the restart routes, the prefactors, the velocity case, the HCN case and the version history before it formats. scripts/test_run.py --checkout runs the example program of Listing S1 in a clone of the bundle. evidence/e1_restart.py reruns the eOn band, scripts/run_restart_walls.py the wall times of Table 7, and scripts/file_sizes.py the sizes of Table 9. For the performance records, bench/make_payloads.py writes the timed files, bench/r0174.py the provenance record, bench/cpp_reads.py builds and checks the C++ programs, and bench/reads.py, bench/memray_peaks.py and bench/cachegrind.py write the timing, heap and Cachegrind records; bench/build_chemfiles.sh and then bench/h2h.py write the head-to-head records, and bench/settle_period.py and bench/binary_formats.py the records of the native readers. For the conformance records, scripts/corpus31_results.py runs examples/corpus31_report of readcon-core over the CON 3.1 corpus, scripts/corpus_paths.py

reads both corpora through the readcon module and through the C library that `cargo build --release` builds and that `cargo build --release --features parallel` builds into `target/clib-parallel`, `scripts/every_path_count.py` runs the reader-path tests of readcon-core and counts their inputs, `scripts/corpus_crosscheck.py` runs the three readers of Table S10, `scripts/referee_probes.py` runs the probes of `paper/cpc/data/referee_probes.json`, `scripts/mutant_differential.py` reads the mutants of Section S4 with the independent reader and the readcon module, and `scripts/clause_coverage.py` lists what names each clause. Listing S3 gives the command line of each of these performance and conformance scripts, the environment variables it reads with their defaults, and the command lines it runs for each cell, file or build, among them those of memray, Valgrind and cargo. The comparison of the earlier releases of CON 3.1 reads each of four published wheels in its own process: `scripts/spec_version_drift.py` takes the readcon wheels of 0.17.0, 0.17.1, 0.17.2 and 0.17.3 and rewrites `data/spec/version_drift.json`.

Listing S2: Rebuild of the paper in a checkout of the untagged commit.

```
git clone readcon-core.bundle readcon-core
cd readcon-core
git checkout f2cea7e88390cd6d295e0f455d8faf2a3fcdc68e
tar -xzf ../paper-cpc.tar.gz
conda create -n readcon-cpc -{}-file paper/cpc/environment/conda-linux-64.txt
conda activate readcon-cpc
python3 -m pip install -{}-no-deps -r paper/cpc/environment/requirements.txt
python3 -m pip install .
python3 paper/cpc/scripts/regen_paper.py
bash paper/cpc/scripts/build_tex.sh    # or bash paper/cpc/scripts/build_tex_tectonic.sh
python3 paper/cpc/scripts/trace_numbers.py -{}-check
```

Listing S3: The commands of the performance and conformance records, run from the checkout of Listing S2. The comment after a command names the task of `paper/cpc/pixi.toml` that runs it. The comments at the top and under each command give the environment variables that the script reads, each with its default, and the command lines that the script runs for each cell, file or build.

```
# NAME=value: a variable the script reads, with its default; after env, the environment
# of the command that follows. $ENV is the environment of the python3 that runs the
# script, $TMP a temporary directory the script makes. Each bench/*.py script below
# reads BENCH_RESULTS=paper/cpc/bench/results/r0174, the directory of its records;
# make_payloads.py, cpp_reads.py, cachegrind.py, h2h.py and binary_formats.py read
# READCON_SRC=., the readcon-core tree whose files they read and build. reads.py,
# memray_peaks.py, cachegrind.py, h2h.py and binary_formats.py run their cells through
# bench/interleave.py, which reads BENCH_SEED=20261104 and BENCH_ROUNDS=11: one untimed
# round and then BENCH_ROUNDS rounds, each a new shuffle of the cells by that seed, each
# run of a cell a new process.
```

```
python3 paper/cpc/bench/make_payloads.py                        # cpc-bench-payloads
#   BENCH_WORK=paper/cpc/bench/work/r0174 BENCH_N_FRAMES=10000 BENCH_SIZE_ROWS=1e6
#   BENCH_SIZES=1e3,1e4,1e5,1e6
python3 paper/cpc/bench/r0174.py                                # cpc-bench-provenance
python3 paper/cpc/bench/cpp_reads.py                            # cpc-bench-cpp
#   RC_PAYLOADS=paper/cpc/bench/work/r0174 CHEMFILES_PREFIX=$ENV
#   BENCH_NATIVE_OUT=paper/cpc/bench/work/native0174
#   CXX=$ENV/bin/x86_64-conda-linux-gnu-c++;
#   env CARGO_TARGET_DIR=$BENCH_NATIVE_OUT/target-capi cargo build -{}-release -{}-lib \
#     -{}-features parallel -{}-manifest-path $READCON_SRC/Cargo.toml, then builds
#   readcon_read and chemfiles_read with $CXX -O3 -std=c++17 -DNDEBUG and runs each of
#   the five cells once, as reads.py does, against the checksums of payloads.json
python3 paper/cpc/bench/reads.py                                # cpc-bench-reads
#   RC_PAYLOADS=paper/cpc/bench/work/r0174; per run of a Python cell:
#   python3 paper/cpc/bench/bench_child.py READER THREADS
#   and per run of a C++ cell, with RAYON_NUM_THREADS, OMP_NUM_THREADS,
#   OPENBLAS_NUM_THREADS and MKL_NUM_THREADS at THREADS:
#   taskset -c CORES $BENCH_NATIVE_OUT/PROGRAM CONTRACT $RC_PAYLOADS/PAYLOAD \
#     $RC_PAYLOADS/WARMUP THREADS
#   CORES is 0 for a cell at one thread and 0-7 for a cell at eight; a run prints READY,
#   and the line that starts its clock follows once the payload is in the page cache
python3 paper/cpc/bench/memray_peaks.py                         # cpc-bench-memray
#   RC_PAYLOADS=paper/cpc/bench/work/r0174; per run of a cell:
#   python3 paper/cpc/bench/memray_peaks.py -{}-child READER THREADS $TMP/read.bin
#   reads the warm-up file of READER, then its payload inside memray.Tracker
python3 paper/cpc/bench/cachegrind.py                           # cpc-bench-cachegrind
#   in $READCON_SRC: env CARGO_TARGET_DIR=target/cachegrind cargo build -{}-release \
#     -{}-example cachegrind_harness -{}-features chemfiles-from-sources; per run of each
#   scenario NAME that the program lists:
#   valgrind -{}-tool=cachegrind -{}-cachegrind-out-file=$TMP/NAME.out -{}-branch-sim=no \
#     -{}-error-exitcode=0 target/cachegrind/release/examples/cachegrind_harness NAME
bash paper/cpc/bench/build_chemfiles.sh                         # cpc-bench-chemfiles
#   CHEMFILES_BUILDS=paper/cpc/bench/work/chemfiles CHEMFILES_JOBS=4 GCC=/usr/bin/gcc
#   GXX=/usr/bin/g++ CLANG=/usr/bin/clang CLANGXX=/usr/bin/clang++
python3 paper/cpc/bench/h2h.py                                  # cpc-bench-h2h
#   -{}-steps payloads,python,cpp by default;
#   BENCH_WORK=paper/cpc/bench/work/r0174 (its files go to $BENCH_WORK/h2h),
#   CHEMFILES_BUILDS as for build_chemfiles.sh, CARGO_TARGET_DIR=$BENCH_WORK/h2h/target
#   GXX=/usr/bin/g++ CLANGXX=/usr/bin/clang++;
#   per campaign, with its CON file, its .rcbin image RCBIN and its peer files in PEERS,
#   per run of each call NAME:
#   python3 paper/cpc/bench/h2h_child.py NAME CON RCBIN PEERS 10000
#   and per run of each call NAME whose peak it records:
#   python3 -m memray run -q -o $TMP/NAME.bin -f -{}-native $TMP/NAME.py
#   the C++ step runs cargo build -{}-release -{}-features parallel -{}-lib in $READCON_SRC,
#   builds $TMP/cpp_ratio_gccO2, _gccO3, _clangO2 and _clangO3 with the build_cell of
#   $READCON_SRC/benches/cpp_peer_matrix.py, and per run of each cell:
#   BINARY MODE FILE
python3 paper/cpc/bench/binary_formats.py                       # cpc-bench-binary-formats
#   -{}-steps build,payloads,check,time by default; BENCH_WORK as for h2h.py
#   (its files go to $W=$BENCH_WORK/binary_formats), GCC=/usr/bin/gcc GXX=/usr/bin/g++;
#   build fetches the sources of build.json, checks their SHA-256, runs the compiler
#   commands that build.json lists and, in $W/readcon_read with the Cargo.lock of
#   $READCON_SRC, env CARGO_TARGET_DIR=$W/target cargo build -{}-release; payloads runs
#   $W/build/bin/gen DIR, then for each CON file and its decimals D (roundtrip, 6, 2)
#   readcon_read write-con REF BOX FILE D and readcon_read write-rcbin CON RCBIN, twice;
#   check runs DRIVER dump MODE THREADS FILE POS BOX for each cell, and time runs
```

```
#   DRIVER time MODE THREADS FILE per run of each cell, after interleave.settle for
#   the settle_ms of $BENCH_RESULTS/binary_formats/settle.json
python3 paper/cpc/bench/settle_period.py                        # cpc-bench-settle
#   -{}-record $BENCH_RESULTS/binary_formats/timings.json by default; BENCH_WORK and
#   READCON_SRC as for binary_formats.py, whose readcon_read it hashes; BENCH_ROUNDS
#   trials; binary_formats.py -{}-steps time then times the cells after the new period

cargo build -{}-release -{}-features zstd,parallel -{}-example corpus31_report
python3 paper/cpc/scripts/corpus31_results.py                   # cpc-corpus31
#   target/release/examples/corpus31_report [-{}-threads N] paper/cpc/data/corpus31
#   at the default thread count and at N = 1, 2 and 8
cargo build -{}-release                                 # cpc-clib, then cpc-clib-parallel:
env CARGO_TARGET_DIR=target/clib-parallel cargo build -{}-release -{}-features parallel
python3 paper/cpc/scripts/corpus_paths.py                       # cpc-corpus-paths
#   READCON_LIB=target/release/libreadcon_core.so
#   READCON_LIB_PARALLEL=target/clib-parallel/release/libreadcon_core.so; for BUILD
#   wheel, clib and clib_parallel, the module and the two libraries:
#   python3 paper/cpc/scripts/corpus_paths.py -{}-child BUILD $TMP/jobs.json
python3 paper/cpc/scripts/every_path_count.py                   # cpc-every-path
#   for each test file NAME that declares mod every_path:
#   env CARGO_TARGET_DIR=target/every_path EVERY_PATH_LOG=$TMP/NAME.log \
#     cargo test -{}-release -{}-features zstd,parallel -{}-test NAME
python3 paper/cpc/scripts/corpus_crosscheck.py                  # cpc-crosscheck
#   for each file: python3 paper/cpc/scripts/corpus_crosscheck.py -{}-child read_con FILE,
#   with -{}-rewrite where its entry marks the rewrite, and -{}-child iter_con FILE
env CARGO_TARGET_DIR=target/cli cargo build -{}-release -{}-features zstd \
  -{}-bin readcon-core                                            # cpc-cli
python3 paper/cpc/scripts/referee_probes.py -{}-check             # cpc-probes
#   READCON_CLI=target/cli/release/readcon-core
#   READCON_LIB=target/release/libreadcon_core.so; -{}-check compares the reads with
#   paper/cpc/data/referee_probes.json and writes nothing, and the run without -{}-check
#   writes that record. It runs $READCON_CLI FILE [OUT] in $TMP, and each read of a
#   probe that has a time and memory limit in a child:
#   python3 paper/cpc/scripts/referee_probes.py -{}-child READER
python3 paper/cpc/scripts/mutant_differential.py                # cpc-mutants
#   -{}-seeds 1-7 -{}-cases 3000 by default; reads each mutant in this process with
#   indep_reader.read_text and readcon.read_con_string
python3 paper/cpc/scripts/clause_coverage.py                    # cpc-clauses
#   reads docs/orgmode/spec.org, both corpus manifests, the probes of
#   paper/cpc/data/referee_probes.json and the tests under tests/ and src/
python3 paper/cpc/scripts/bench_readers.py                      # reader exchange
#   LARGE_FRAMES=100000. The script writes paper/cpc/logs/reader-exchange.log.
#   It sets these variables in the child of every arm, from the checkout:
#   READCON_BASE=paper/cpc/build/baseline-readcon
#   READCON_V0173=paper/cpc/build/readcon-0.17.3
#   EON_PKG=paper/cpc/bench/work/peers/eon
#   EON_ALLFRAMES=paper/cpc/bench/work/eon_allframes
#   READCON_BASE is the baseline module on PYTHONPATH for readcon_base.
#   READCON_V0173 is readcon 0.17.3, built from
#   commit 12ca800bc2212c565941e04094cba6b4d99a62e0.
#   READCON_BASE is the baseline module, built from
#   commit ac4f437989dea5aad9f75ff045af09ccf7127eba.
#   The tip arms import the module built from the measurement commit:
#   MATURIN_PEP517_ARGS="--features python" \
#     python3 -m pip install --no-build-isolation .
#   The 0.17.3 arm uses that command in a worktree of
#   12ca800bc2212c565941e04094cba6b4d99a62e0, then
```

```
#   --target paper/cpc/build/readcon-0.17.3.
#   The baseline arm uses --target paper/cpc/build/baseline-readcon.
#   Public tag v0.18.0 is commit
#   d5e77f997438557c72c3b3e360d446310f85d6b7.
#   It installs to paper/cpc/build/readcon-0.18.0.
#   BENCH_VARIANT=public selects that module for
#   readcon_full_t1, readcon_full_t8, readcon_pos_t1
#   and readcon_pos_t8.
#   EON_PKG is the eOn tree whose eon.fileio.loadcons the eon_py arm imports.
#   EON_ALLFRAMES is the binary built from paper/cpc/bench/eon_allframes.cpp.
#   Before the samples the script runs, from the repository root:
#   /usr/bin/g++ -std=c++20 -O3 -DNDEBUG -pthread -DQUILL_DISABLE_NON_PREFIXED_MACROS
#     -o paper/cpc/bench/work/eon_allframes paper/cpc/bench/eon_allframes.cpp
#     -Ipaper/cpc/bench/work/peers/eon/include
#     -Ipaper/cpc/bench/work/peers/eon/client/thirdparty
#     -Ipaper/cpc/bench/work/peers/eon/subprojects/quill-11.1.0/include
#     -Ipaper/cpc/bench/work/peers/eon/subprojects/inih-r62/cpp
#     -Ipaper/cpc/bench/work/peers/eon/subprojects/nlohmann_json-3.12.0/single_include
#     -Ipaper/cpc/bench/work/peers/eon/subprojects/highway-1.4.0
#     -Ipaper/cpc/bench/work/peers/eon/subprojects/rgpot/third_party/vesin
#     -Ipaper/cpc/bench/work/peers/eon/subprojects/readcon-core/include
#     -Ipaper/cpc/bench/work/peers/eon/subprojects/rgpot/CppCore
#     -Ipaper/cpc/bench/work/peers/eon/subprojects/rgpot/include
#     -Ipaper/cpc/bench/work/peers/eon/subprojects/rgpot/CppCore/rgpot/rpc
#     -Ipaper/cpc/bench/work/peers/eon/subprojects/rgpot/third_party/lepton/include
#     -I/usr/include/eigen3
#     -Lpaper/cpc/bench/work/peers/eon/build-h2h/subprojects/readcon-core
#     -Wl,--disable-new-dtags
#     -Wl,-rpath,$ORIGIN/peers/eon/build-h2h/subprojects/readcon-core
#     -leonclib -lreadcon_core -lsocket_nwchem -lemt -lasap -lextpot
#     -lrgpot_pot -llammps_pot -l:librgpot.so.4
#   Each arm is one fresh process, six times, interleaved. The child command is
#   python3 -c '<worker in bench_readers.py>' ARM FILE 100000 32
#   and the script sets OMP_NUM_THREADS, OPENBLAS_NUM_THREADS, MKL_NUM_THREADS,
#   NUMEXPR_NUM_THREADS and RAYON_NUM_THREADS to the thread count of that arm.
#   readcon_full_t1: readcon.read_con(FILE, 1) on frames.con
#   readcon_full_t8: readcon.read_con(FILE, 8) on frames.con
#   readcon_pos_t1: readcon.read_positions(FILE, 1) on frames.con
#   readcon_pos_t8: readcon.read_positions(FILE, 8) on frames.con
#   readcon_base: readcon.read_con(FILE, 1) with READCON_BASE on PYTHONPATH
#   readcon_v0173: readcon.read_con(FILE, 1) with READCON_V0173 on PYTHONPATH
#   readcon_first: readcon.read_first_frame(FILE), one frame
#   readcon_db: ConCorpus.append_trajectory_str then get_positions, an ingest
#   ase_eon: ase.io.read(FILE, index=':', format='eon') then get_positions
#   eon: Matter.con2matter(FILE), the first frame only
#   eon_py: eon.fileio.loadcons(FILE), every frame, with EON_PKG on PYTHONPATH
#   readcon_eon_api: $EON_ALLFRAMES FILE, read_all_frames then con2matter
#   conbin: readcon.read_conbin on frames.conbin
#   numpy: numpy.loadtxt usecols on frames.xyz
#   chemfiles: chemfiles.Trajectory positions on frames.xyz
#   ovito: pipeline.compute particles.positions on frames.xyz
#   ase_extxyz: ase.io.read then get_positions on frames.xyz
#   pymatgen: XYZ.from_file cart_coords on frames.xyz
#   h5py: the position dataset of frames.h5md
#   mdanalysis: Universe.atoms.positions on frames.h5md
```

S12. Generated records

The binary-format, DLPack, ordering and head-to-head tables, and the layout figures, are generated with the records they name.

Native reader	Read	CON	CON, six decimals	CON, two decimals	`.rcbin`
xdrfile, XTC	by frame	5.0	3.9	3.5	–
	all frames	5.2	4.3	4.1	1.3
xdrfile, TRR	by frame	3.4	2.7	2.4	–
	all frames	3.7	3.1	2.9	0.95
molfile, DCD	by frame	37	29	26	–
	all frames	20	17	16	5.2
TNG	by frame	0.52	0.41	0.37	–
	all frames	1.6	1.3	1.2	0.40

Reader	File	Read	Threads	Median (s)	IQR (s)	Frames/s	Error (Å)
readcon-core	CON	by frame	1	1.098	0.014	9.1	0
readcon-core	CON, six decimals	by frame	1	0.864	0.012	11.6	5.0×10^{-7}
readcon-core	CON, two decimals	by frame	1	0.776	0.010	12.9	5.0×10^{-3}
readcon-core	CON	all frames	1	1.339	0.016	7.5	0
readcon-core	CON, six decimals	all frames	1	1.107	0.034	9.0	5.0×10^{-7}
readcon-core	CON, two decimals	all frames	1	1.035	0.081	9.7	5.0×10^{-3}
readcon-core	CON	all frames	8	0.688	0.027	14.5	0
readcon-core	CON, six decimals	all frames	8	0.526	0.011	19.0	5.0×10^{-7}
readcon-core	CON, two decimals	all frames	8	0.466	0.007	21.4	5.0×10^{-3}
readcon-core	CON	columns	1	1.111	0.010	9.0	0
readcon-core	CON, six decimals	columns	1	0.819	0.008	12.2	5.0×10^{-7}
readcon-core	CON, two decimals	columns	1	0.714	0.015	14.0	5.0×10^{-3}
readcon-core	`.rcbin`	image	1	0.340	0.018	29.4	0
xdrfile	XTC	by frame	1	0.219	0.001	45.6	5.0×10^{-3}
xdrfile	XTC	all frames	1	0.255	0.003	39.2	5.0×10^{-3}
xdrfile	TRR	by frame	1	0.325	0.008	30.7	9.5×10^{-6}
xdrfile	TRR	all frames	1	0.359	0.007	27.9	9.5×10^{-6}
libxtc[a]	XTC	by frame	1	0.092	0.001	109	505
libxtc[a]	XTC	all frames	1	0.126	0.006	79.1	505
libxtc[a]	XTC	by frame	8	0.044	0.002	225	505
libxtc[a]	XTC	all frames	8	0.054	0.005	185	505
molfile	DCD	by frame	1	0.030	0.001	333	1.5×10^{-5}
molfile	DCD	all frames	1	0.066	0.003	152	1.5×10^{-5}
TNG	TNG	by frame	1	2.115	0.047	4.7	5.0×10^{-3}
TNG	TNG	all frames	1	0.852	0.034	11.7	5.0×10^{-3}

atoms	exchange median (s)	exchange type 7 quartiles (s)	copy median (s)	copy type 7 quartiles (s)
32	1.629×10^{-6}	$[1.500 \times 10^{-6}, 2.776 \times 10^{-6}]$	1.207×10^{-6}	$[1.045 \times 10^{-6}, 1.691 \times 10^{-6}]$
128	1.602×10^{-6}	$[1.497 \times 10^{-6}, 1.876 \times 10^{-6}]$	1.498×10^{-6}	$[1.388 \times 10^{-6}, 1.842 \times 10^{-6}]$
512	1.462×10^{-6}	$[1.401 \times 10^{-6}, 1.900 \times 10^{-6}]$	2.188×10^{-6}	$[2.000 \times 10^{-6}, 2.395 \times 10^{-6}]$
2048	1.721×10^{-6}	$[1.677 \times 10^{-6}, 2.086 \times 10^{-6}]$	2.983×10^{-6}	$[2.831 \times 10^{-6}, 3.179 \times 10^{-6}]$
8192	1.415×10^{-6}	$[1.320 \times 10^{-6}, 1.765 \times 10^{-6}]$	5.524×10^{-6}	$[5.360 \times 10^{-6}, 5.990 \times 10^{-6}]$
32768	4.002×10^{-6}	$[3.819 \times 10^{-6}, 4.591 \times 10^{-6}]$	3.238×10^{-5}	$[3.176 \times 10^{-5}, 3.292 \times 10^{-5}]$

		chemfiles		readcon-core columns			readcon-core by frame		
Compiler	File	Median (s)	IQR (s)	Median (s)	IQR (s)	Ratio	Median (s)	IQR (s)	Ratio
coordinates									
g++ -O2	XYZ	0.3719	0.0030	0.2319	0.0099	0.62	0.2915	0.0075	0.78
g++ -O3	XYZ	0.3277	0.0032	0.2263	0.0058	0.69	0.2922	0.0109	0.89
clang++ -O2	XYZ	0.3595	0.0024	0.2306	0.0043	0.64	0.2995	0.0050	0.83
clang++ -O3	XYZ	0.3406	0.0020	0.2336	0.0080	0.69	0.2912	0.0076	0.85
velocities									
g++ -O2	XYZ	0.3741	0.0048	0.4489	0.0021	1.2	0.5120	0.0031	1.4
g++ -O2	GRO	0.5314	0.0034	0.4788	0.0098	0.90	0.5195	0.0048	0.98
g++ -O3	XYZ	0.3286	0.0019	0.4496	0.0068	1.4	0.5139	0.0021	1.6
g++ -O3	GRO	0.4744	0.0051	0.4812	0.0035	1.0	0.5188	0.0042	1.1
clang++ -O2	XYZ	0.3630	0.0012	0.4466	0.0057	1.2	0.5130	0.0068	1.4
clang++ -O2	GRO	0.4788	0.0027	0.4778	0.0039	1.0	0.5173	0.0042	1.1
clang++ -O3	XYZ	0.3431	0.0034	0.4477	0.0044	1.3	0.5109	0.0043	1.5
clang++ -O3	GRO	0.4769	0.0009	0.4759	0.0043	1.0	0.5165	0.0056	1.1

Call	Median (s)	IQR (s)	Peak heap (MB)
coordinates			
readcon `read_con`	0.2205	0.0038	413.379
readcon `read_arrays`	0.2122	0.0097	220.825
readcon `read_arrays`, low memory	0.3021	0.0076	58.351
readcon `read_rcbin`	0.0144	0.0002	57.697
readcon `read_rcbin`, caller buffer	0.0099	0.0003	–
h5py `position/value`	0.0158	0.0002	60.044
ASE `.traj`	0.8049	0.0096	153.400
ASE extended XYZ	2.8437	0.0147	117.378
velocities			
readcon `read_con`	0.5036	0.0209	992.440
readcon `read_arrays`	0.4123	0.0016	259.214
readcon `read_arrays`, low memory	0.5366	0.0069	58.351
readcon `read_rcbin`	0.0124	0.0002	57.697
readcon `read_rcbin`, caller buffer	0.0081	0.0001	–
h5py `position/value`	0.0159	0.0003	60.044
ASE `.traj`	1.0405	0.0191	207.091
ASE extended XYZ	2.8445	0.0177	117.378

Table S24: The exact two-sided Mann–Whitney test of every timing ratio of Section 8, readcon-core time over peer time, on the 11 runs of each cell, each run a fresh process. The ratio is of the medians, as in the table of each heading. The HL ratio is the Hodges–Lehmann estimate with its distribution-free interval, of coverage at least 0.952. Holm rejects a claim at the family-wise level 0.05 over all 75 claims.

Claim, readcon-core / peer	Ratio	HL ratio [interval]	p	Holm
Python reads (Table S27), one thread				
readcon-core / ASE, extended XYZ	0.457	0.456 [0.452, 0.460]	2.8×10^{-6}	reject
readcon-core / `extxyz` (C)	1.79	1.78 [1.77, 1.80]	2.8×10^{-6}	reject
readcon-core / OVITO, extended XYZ	0.532	0.534 [0.514, 0.555]	2.8×10^{-6}	reject
readcon-core / ASE trajectory	1.84	1.82 [1.80, 1.85]	2.8×10^{-6}	reject
readcon-core / h5py, H5MD	16.4	16.4 [16.2, 16.6]	2.8×10^{-6}	reject
readcon-core / MDAnalysis, H5MD	0.462	0.461 [0.456, 0.469]	2.8×10^{-6}	reject
readcon-core, no velocities / ASE `eon`	0.405	0.404 [0.401, 0.408]	2.8×10^{-6}	reject
readcon-core, no velocities / chemfiles, XYZ	0.191	0.191 [0.189, 0.192]	2.8×10^{-6}	reject
readcon-core, no velocities / OVITO, XYZ	0.586	0.590 [0.570, 0.612]	2.8×10^{-6}	reject
readcon-core, one thread / eight	1.09	1.09 [1.06, 1.10]	2.8×10^{-6}	reject
readcon-core, no velocities, one thread / eight	1.18	1.17 [1.16, 1.19]	2.8×10^{-6}	reject
readcon-core, frames only, one thread / eight	1.93	1.94 [1.87, 1.99]	2.8×10^{-6}	reject
C++ reads (Table S26)				
readcon-core / chemfiles, extended XYZ	0.695	0.694 [0.688, 0.701]	2.8×10^{-6}	reject
readcon-core, no velocities / chemfiles, XYZ	0.992	0.994 [0.981, 1.02]	0.52	keep
readcon-core, one thread / eight	1.65	1.65 [1.61, 1.68]	2.8×10^{-6}	reject
Python head-to-head (Table S23)				
coordinates: `read_arrays` / ASE extended XYZ	0.0746	0.0746 [0.0737, 0.0773]	2.8×10^{-6}	reject
coordinates: `read_arrays` / ASE `.traj`	0.264	0.264 [0.260, 0.273]	2.8×10^{-6}	reject
coordinates: `read_arrays` / h5py	13.4	13.5 [13.3, 13.9]	2.8×10^{-6}	reject
coordinates: `read_rcbin` / h5py	0.911	0.910 [0.895, 0.921]	2.8×10^{-6}	reject

Claim, readcon-core / peer	Ratio	HL ratio [interval]	p	Holm
velocities: `read_arrays` / ASE extended XYZ	0.145	0.145 [0.144, 0.145]	2.8×10^{-6}	reject
velocities: `read_arrays` / ASE `.traj`	0.396	0.397 [0.394, 0.401]	2.8×10^{-6}	reject
velocities: `read_arrays` / h5py	25.9	25.9 [25.6, 26.1]	2.8×10^{-6}	reject
velocities: `read_rcbin` / h5py	0.781	0.781 [0.772, 0.791]	2.8×10^{-6}	reject
C++ head-to-head at one thread (Table S22)				
coordinates, g++ -O2: columns / XYZ	0.624	0.626 [0.607, 0.635]	2.8×10^{-6}	reject
coordinates, g++ -O2: by frame / XYZ	0.784	0.786 [0.770, 0.796]	2.8×10^{-6}	reject
coordinates, g++ -O3: columns / XYZ	0.690	0.693 [0.684, 0.703]	2.8×10^{-6}	reject
coordinates, g++ -O3: by frame / XYZ	0.891	0.892 [0.883, 0.916]	2.8×10^{-6}	reject
coordinates, clang++ -O2: columns / XYZ	0.641	0.642 [0.632, 0.648]	2.8×10^{-6}	reject
coordinates, clang++ -O2: by frame / XYZ	0.833	0.832 [0.818, 0.844]	2.8×10^{-6}	reject
coordinates, clang++ -O3: columns / XYZ	0.686	0.681 [0.661, 0.688]	2.8×10^{-6}	reject
coordinates, clang++ -O3: by frame / XYZ	0.855	0.853 [0.840, 0.863]	2.8×10^{-6}	reject
velocities, g++ -O2: columns / XYZ	1.20	1.20 [1.18, 1.21]	2.8×10^{-6}	reject
velocities, g++ -O2: by frame / XYZ	1.37	1.37 [1.35, 1.38]	2.8×10^{-6}	reject
velocities, g++ -O2: columns / GRO	0.901	0.904 [0.896, 0.915]	2.8×10^{-6}	reject
velocities, g++ -O2: by frame / GRO	0.978	0.979 [0.973, 0.986]	1.1×10^{-5}	reject
velocities, g++ -O3: columns / XYZ	1.37	1.37 [1.36, 1.38]	2.8×10^{-6}	reject
velocities, g++ -O3: by frame / XYZ	1.56	1.57 [1.56, 1.58]	2.8×10^{-6}	reject
velocities, g++ -O3: columns / GRO	1.01	1.01 [1.00, 1.02]	0.0032	reject
velocities, g++ -O3: by frame / GRO	1.09	1.09 [1.09, 1.10]	2.8×10^{-6}	reject
velocities, clang++ -O2: columns / XYZ	1.23	1.23 [1.22, 1.24]	2.8×10^{-6}	reject
velocities, clang++ -O2: by frame / XYZ	1.41	1.41 [1.41, 1.43]	2.8×10^{-6}	reject
velocities, clang++ -O2: columns / GRO	0.998	0.998 [0.991, 1.01]	0.44	keep
velocities, clang++ -O2: by frame / GRO	1.08	1.08 [1.07, 1.09]	2.8×10^{-6}	reject
velocities, clang++ -O3: columns / XYZ	1.30	1.30 [1.29, 1.31]	2.8×10^{-6}	reject
velocities, clang++ -O3: by frame / XYZ	1.49	1.49 [1.48, 1.50]	2.8×10^{-6}	reject

Claim, readcon-core / peer	Ratio	HL ratio [interval]	p	Holm
velocities, clang++ -O3: columns / GRO	0.998	0.998 [0.993, 1.00]	0.30	keep
velocities, clang++ -O3: by frame / GRO	1.08	1.08 [1.07, 1.09]	2.8×10^{-6}	reject
Native readers at one thread (Table S19)				
by frame, CON / xdrfile XTC	5.01	5.03 [5.00, 5.09]	2.8×10^{-6}	reject
by frame, CON six / xdrfile XTC	3.94	3.95 [3.91, 3.99]	2.8×10^{-6}	reject
by frame, CON two / xdrfile XTC	3.54	3.55 [3.52, 3.59]	2.8×10^{-6}	reject
all frames, CON / xdrfile XTC	5.25	5.26 [5.20, 5.37]	2.8×10^{-6}	reject
all frames, CON six / xdrfile XTC	4.34	4.37 [4.30, 4.47]	2.8×10^{-6}	reject
all frames, CON two / xdrfile XTC	4.06	4.11 [3.99, 4.32]	2.8×10^{-6}	reject
all frames, `.rcbin` / xdrfile XTC	1.33	1.34 [1.29, 1.37]	2.8×10^{-6}	reject
by frame, CON / xdrfile TRR	3.38	3.37 [3.32, 3.42]	2.8×10^{-6}	reject
by frame, CON six / xdrfile TRR	2.65	2.64 [2.58, 2.67]	2.8×10^{-6}	reject
by frame, CON two / xdrfile TRR	2.39	2.38 [2.32, 2.40]	2.8×10^{-6}	reject
all frames, CON / xdrfile TRR	3.73	3.73 [3.69, 3.79]	2.8×10^{-6}	reject
all frames, CON six / xdrfile TRR	3.08	3.10 [3.04, 3.16]	2.8×10^{-6}	reject
all frames, CON two / xdrfile TRR	2.88	2.91 [2.83, 3.07]	2.8×10^{-6}	reject
all frames, `.rcbin` / xdrfile TRR	0.948	0.948 [0.911, 0.972]	0.0041	reject
by frame, CON / molfile DCD	36.6	36.7 [35.6, 37.2]	2.8×10^{-6}	reject
by frame, CON six / molfile DCD	28.8	28.6 [27.9, 29.2]	2.8×10^{-6}	reject
by frame, CON two / molfile DCD	25.9	25.8 [25.1, 26.3]	2.8×10^{-6}	reject
all frames, CON / molfile DCD	20.3	20.3 [19.6, 20.7]	2.8×10^{-6}	reject
all frames, CON six / molfile DCD	16.8	16.8 [16.3, 17.2]	2.8×10^{-6}	reject
all frames, CON two / molfile DCD	15.7	15.7 [15.3, 16.6]	2.8×10^{-6}	reject
all frames, `.rcbin` / molfile DCD	5.17	5.08 [4.92, 5.28]	2.8×10^{-6}	reject
by frame, CON / TNG	0.519	0.523 [0.516, 0.532]	2.8×10^{-6}	reject
by frame, CON six / TNG	0.408	0.410 [0.402, 0.416]	2.8×10^{-6}	reject
by frame, CON two / TNG	0.367	0.368 [0.362, 0.375]	2.8×10^{-6}	reject
all frames, CON / TNG	1.57	1.58 [1.54, 1.62]	2.8×10^{-6}	reject

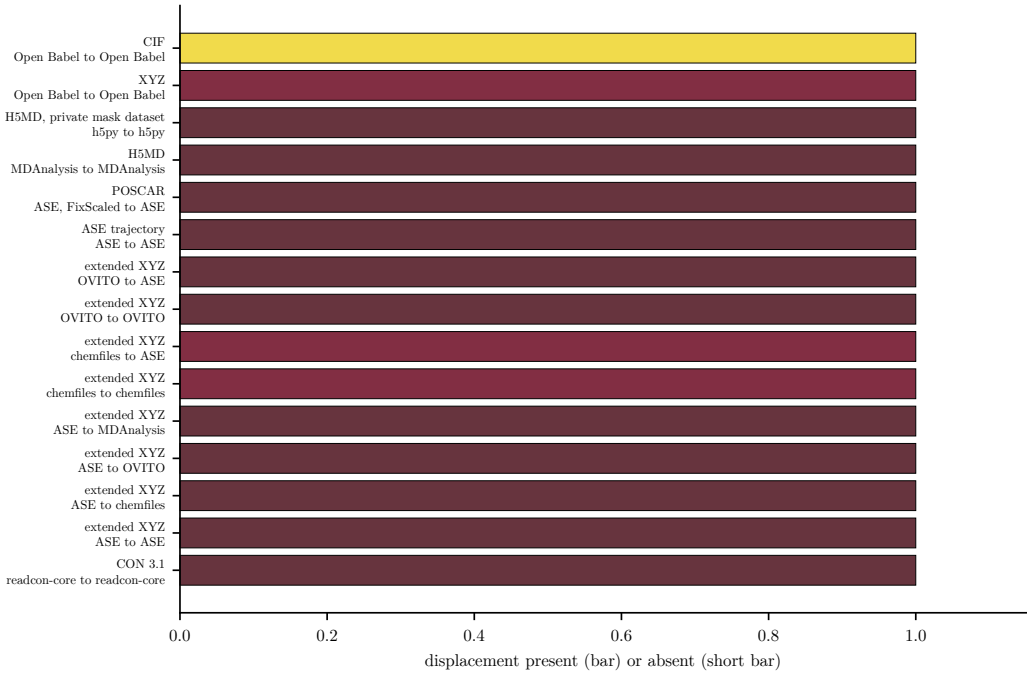
Claim, readcon-core / peer	Ratio	HL ratio [interval]	p	Holm
all frames, CON six / TNG	1.30	1.31 [1.27, 1.35]	2.8×10^{-6}	reject
all frames, CON two / TNG	1.21	1.23 [1.19, 1.30]	2.8×10^{-6}	reject
all frames, `.rcbin` / TNG	0.399	0.398 [0.378, 0.415]	2.8×10^{-6}	reject

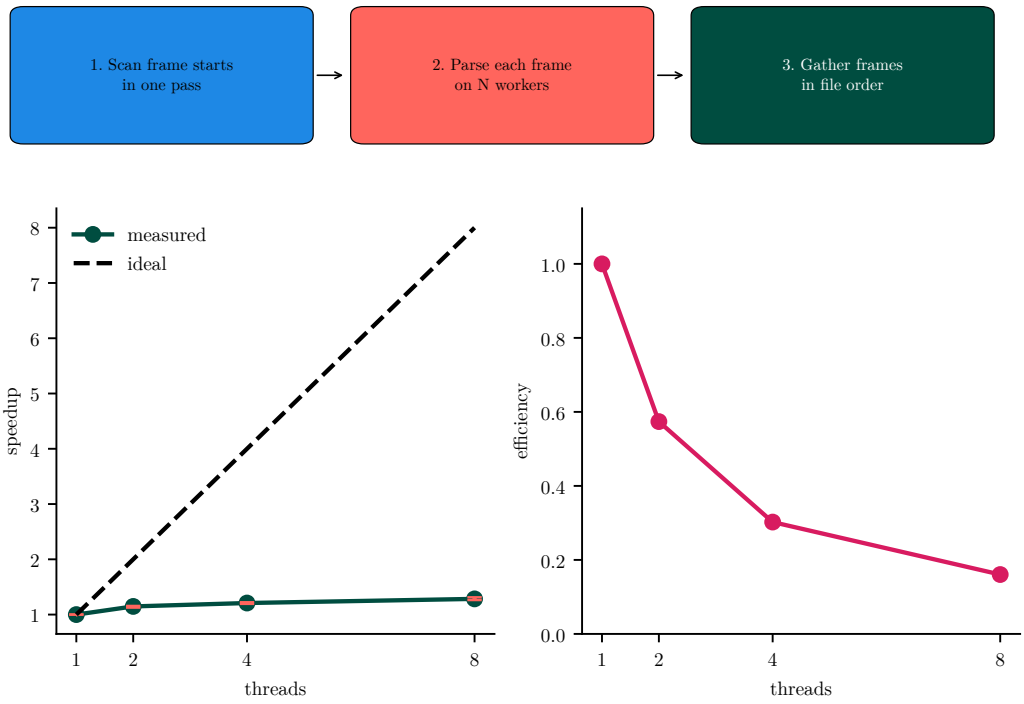
Work	Scenario	Instructions	IQR
parse one 218-atom frame	`parse_cuh2_218`	10,445,209	0
parse 100 frames	`parse_100_frames`	32,470,382	0
skip 100 frames	`forward_100_frames`	7,291,661	0
parse a two-frame file of four atoms	`parse_multi_2x4`	6,454,674	0
skip in that file	`forward_multi_2x4`	4,605,178	0
parse a velocity file	`convel_multi`	7,340,005	0
write 100 frames	`write_100_frames`	21,640,874	0
parse floats, fast-float2	`float_fast_float2`	32,340,278	0
parse floats, Rust standard library	`float_std_parse`	33,190,954	0
chemfiles C++, XYZ file	`chemfiles_xyz_path`	9,277,979	0
chemfiles C++, XYZ in memory	`chemfiles_xyz_memory`	9,111,497	0
chemfiles C++, selection	`chemfiles_select_name_O`	5,616,632	0

Reader	File	Threads	Median (s)	IQR (s)	readcon/ reader
Positions, velocities, masks, identifiers, atomic numbers, cell and periodic flags					
readcon-core C++	CON 3.1	1	1.09	0.018	–
readcon-core C++	CON 3.1	8	0.661	0.026	–
chemfiles C++	extended XYZ	1	1.57	0.017	0.69
Positions and atomic numbers					
readcon-core C++	CON 3.1, no velocities	1	0.554	0.021	–
chemfiles C++	XYZ	1	0.559	0.013	0.99

Reader	File	Threads	Median (s)	IQR (s)	Peak heap (MiB)	readcon/ reader
Positions, velocities, masks, identifiers, atomic numbers, cell and periodic flags						
readcon-core	CON 3.1	1	2.62	0.034	1803	–
readcon-core	CON 3.1	8	2.39	0.11	1801	–
ASE	extended XYZ	1	5.73	0.030	158	0.46
`extxyz` (C)	extended XYZ	1	1.46	0.0100	220	1.8
OVITO	extended XYZ	1	4.92	0.34	242	0.53
ASE	ASE trajectory	1	1.42	0.024	218	1.8
h5py	H5MD	1	0.160	0.0026	141	16
Positions and velocities in binary32						
MDAnalysis	H5MD	1	5.67	0.12	55	–
Positions and atomic numbers						
readcon-core	CON 3.1, no velocities	1	1.35	0.021	1237	–
readcon-core	CON 3.1, no velocities	8	1.15	0.021	1233	–
ASE `eon`	CON 3.1, no velocities	1	3.33	0.031	108	0.40
chemfiles	XYZ	1	7.08	0.054	356	0.19
OVITO	XYZ	1	2.30	0.14	108	0.59
Frames only, no field copied out						
readcon-core, frames only	CON 3.1	1	0.886	0.015	934	–
readcon-core, frames only	CON 3.1	8	0.459	0.025	953	–


CON frame
What a version 1 reader uses
1 generator text
2 JSON metadata, or a comment in v1
3 |a| |b| |c|
4 alpha beta gamma
5 free text
6 free text
7 type count
8 per-type counts
9 masses
symbol, coordinates of component N
x y z flag id
blank line, then a section
Line 2 is only a comment
Column 4 is 0 or 1
1 means fixed on x, y and z
No sections array
No units object
Velocities, if present, use the natural unit


CIF
Open Babel to Open Babel
XYZ
Open Babel to Open Babel
H5MD, private mask dataset
h5py to h5py
H5MD
MDAnalysis to MDAnalysis
POSCAR
ASE, FixScaled to ASE
ASE trajectory
ASE to ASE
extended XYZ
OVITO to ASE
extended XYZ
OVITO to OVITO
extended XYZ
chemfiles to ASE
extended XYZ
chemfiles to chemfiles
extended XYZ
ASE to MDAnalysis
extended XYZ
ASE to OVITO
extended XYZ
ASE to chemfiles
extended XYZ
ASE to ASE
CON 3.1
readcon-core to readcon-core
0.0
0.2
0.4
0.6
0.8
1.0
displacement present (bar) or absent (short bar)



References

[1] R. Goswami, Reproducible orchestration of best practices for reaction path optimization with the nudged elastic band, MethodsX 16 (2026) 103899. doi:10.1016/j.mex.2026.103899.

[2] R. Goswami, Two-dimensional RMSD projections for reaction path visualization and validation, MethodsX 16 (2026) 103851. doi:10.1016/j.mex.2026.103851.

[3] R. Goswami, M. Masterov, S. Kamath, A. Peña-Torres, H. Jónsson, Efficient implementation of gaussian process regression accelerated saddle point searches with application to molecular reactions, Journal of Chemical Theory and Computation 21 (16) (2025) 7935–7943. doi:10.1021/acs.jctc.5c00866.

[4] S. T. Chill, M. Welborn, R. Terrell, L. Zhang, J.-C. Berthet, A. Pedersen, H. Jónsson, G. Henkelman, EON: software for long time

simulations of atomic scale systems, Modelling and Simulation in Materials Science and Engineering 22 (5) (2014) 055002. doi:10.1088/0965-0393/22/5/055002.

[5] J. Kermode, libAtoms/extxyz: v0.4.6, extended XYZ specification and parsing tools, GitHub, software and format specification, version 0.4.6 (2026).
URL https://github.com/libAtoms/extxyz/releases/tag/v0.4.6

[6] D. Spångberg, D. S. D. Larsson, D. van der Spoel, Trajectory NG: portable, compressed, general molecular dynamics trajectories, Journal of Molecular Modeling 17 (10) (2011) 2669–2685. doi:10.1007/s00894-010-0948-5.

[7] M. Lundborg, R. Apostolov, D. Spångberg, A. Gärdenäs, D. van der Spoel, E. Lindahl, An efficient and extensible format, library, and API for binary trajectory data from molecular simulations, Journal of Computational Chemistry 35 (3) (2014) 260–269. doi:10.1002/jcc.23495.

[8] M. J. Abraham, T. Murtola, R. Schulz, S. Páll, J. C. Smith, B. Hess, E. Lindahl, GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers, SoftwareX 1-2 (2015) 19–25. doi:10.1016/j.softx.2015.06.001.

[9] B. R. Brooks, C. L. Brooks, A. D. Mackerell, L. Nilsson, R. J. Petrella, B. Roux, Y. Won, G. Archontis, C. Bartels, S. Boresch, A. Caflisch, L. Caves, Q. Cui, A. R. Dinner, M. Feig, S. Fischer, J. Gao, M. Hodoscek, W. Im, K. Kuczera, T. Lazaridis, J. Ma, V. Ovchinnikov, E. Paci, R. W. Pastor, C. B. Post, J. Z. Pu, M. Schaefer, B. Tidor, R. M. Venable, H. L. Woodcock, X. Wu, W. Yang, D. M. York, M. Karplus, CHARMM: The biomolecular simulation program, Journal of Computational Chemistry 30 (10) (2009) 1545–1614. doi:10.1002/jcc.21287.

[10] D. A. Case, T. E. Cheatham, T. Darden, H. Gohlke, R. Luo, K. M. Merz, A. Onufriev, C. Simmerling, B. Wang, R. J. Woods, The Amber biomolecular simulation programs, Journal of Computational Chemistry 26 (16) (2005) 1668–1688. doi:10.1002/jcc.20290.

[11] The Amber Developers, AMBER NetCDF trajectory/restart convention, version 1.0, revision C, https://ambermd.org/netcdf/nctraj.xhtml, accessed 2026-10-31 (2026).

[12] R. Rew, G. Davis, NetCDF: an interface for scientific data access, IEEE Computer Graphics and Applications 10 (4) (1990) 76–82. doi:10.1109/38.56302.

[13] H. M. Berman, J. Westbrook, Z. Feng, G. Gilliland, T. N. Bhat, H. Weissig, I. N. Shindyalov, P. E. Bourne, The protein data bank, Nucleic Acids Research 28 (1) (2000) 235–242. doi:10.1093/nar/28.1.235.

[14] H. Berman, K. Henrick, H. Nakamura, Announcing the worldwide protein data bank, Nature Structural & Molecular Biology 10 (12) (2003) 980–980. doi:10.1038/nsb1203-980.

[15] J. D. Westbrook, P. E. Bourne, STAR/mmCIF: An ontology for macromolecular structure, Bioinformatics 16 (2) (2000) 159–168. doi:10.1093/bioinformatics/16.2.159.

[16] A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott, S. J. Plimpton, LAMMPS – a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales, Computer Physics Communications 271 (2022) 108171. doi:10.1016/j.cpc.2021.108171.

[17] P. Hirel, Atomsk: A tool for manipulating and converting atomic data files, Computer Physics Communications 197 (2015) 212–219. doi:10.1016/j.cpc.2015.07.012.

[18] N. M. O'Boyle, A. L. Tenderholt, K. M. Langner, cclib: A library for package-independent computational chemistry algorithms, Journal of Computational Chemistry 29 (5) (2008) 839–845. doi:10.1002/jcc.20823.

[19] T. Verstraelen, W. Adams, L. Pujal, A. Tehrani, B. D. Kelly, L. Macaya, F. Meng, M. Richer, R. Hernández-Esparza, X. D. Yang, M. Chan, T. D. Kim, M. Cools-Ceuppens, V. Chuiko, E. Vöhringer-Martinez, P. W. Ayers, F. Heidar-Zadeh, IOData: A Python library for reading, writing, and converting computational chemistry file formats and generating

input files, Journal of Computational Chemistry 42 (6) (2021) 458–464. doi:10.1002/jcc.26468.

[20] E. Posenitskiy, V. G. Chilkuri, A. Ammar, M. Hapka, K. Pernal, R. Shinde, E. J. Landinez Borda, C. Filippi, K. Nakano, O. Kohulák, S. Sorella, P. de Oliveira Castro, W. Jalby, P. L. Ríos, A. Alavi, A. Scemama, TREXIO: A file format and library for quantum chemistry, The Journal of Chemical Physics 158 (17) (2023) 174801. doi:10.1063/5.0148161.

[21] D. G. A. Smith, D. Altarawy, L. A. Burns, M. Welborn, L. N. Naden, L. Ward, S. Ellis, B. P. Pritchard, T. D. Crawford, The MolSSI QCArchive project: An open-source platform to compute, organize, and share quantum chemistry data, WIREs Computational Molecular Science 11 (2) (2021) e1491. doi:10.1002/wcms.1491.

[22] C. W. Andersen, R. Armiento, E. Blokhin, G. J. Conduit, S. Dwaraknath, M. L. Evans, Á. Fekete, A. Gopakumar, S. Gražulis, A. Merkys, F. Mohamed, C. Oses, G. Pizzi, G.-M. Rignanese, M. Scheidgen, L. Talirz, C. Toher, D. Winston, R. Aversa, K. Choudhary, P. Colinet, S. Curtarolo, D. Di Stefano, C. Draxl, S. Er, M. Esters, M. Fornari, M. Giantomassi, M. Govoni, G. Hautier, V. Hegde, M. K. Horton, P. Huck, G. Huhs, J. Hummelshøj, A. Kariryaa, B. Kozinsky, S. Kumbhar, M. Liu, N. Marzari, A. J. Morris, A. A. Mostofi, K. A. Persson, G. Petretto, T. Purcell, F. Ricci, F. Rose, M. Scheffler, D. Speckhard, M. Uhrin, A. Vaitkus, P. Villars, D. Waroquiers, C. Wolverton, M. Wu, X. Yang, OPTIMADE, an API for exchanging materials data, Scientific Data 8 (1) (2021) 217. doi:10.1038/s41597-021-00974-z.

[23] C. Draxl, M. Scheffler, The NOMAD laboratory: from data sharing to artificial intelligence, Journal of Physics: Materials 2 (3) (2019) 036001. doi:10.1088/2515-7639/ab13bb.

[24] K. T. Winther, M. J. Hoffmann, J. R. Boes, O. Mamun, M. Bajdich, T. Bligaard, Catalysis-Hub.org, an open electronic structure database for surface reactions, Scientific Data 6 (1) (2019) 75. doi:10.1038/s41597-019-0081-y.

[25] M. Schreiner, A. Bhowmik, T. Vegge, J. Busk, O. Winther, Transition1x – a dataset for building generalizable reactive machine learning potentials, Scientific Data 9 (1) (2022) 779. doi:10.1038/s41597-022-01870-w.

[26] L. Chanussot, A. Das, S. Goyal, T. Lavril, M. Shuaibi, M. Riviere, K. Tran, J. Heras-Domingo, C. Ho, W. Hu, A. Palizhati, A. Sriram, B. Wood, J. Yoon, D. Parikh, C. L. Zitnick, Z. Ulissi, Open catalyst 2020 (OC20) dataset and community challenges, ACS Catalysis 11 (10) (2021) 6059–6072. doi:10.1021/acscatal.0c04525.

[27] R. T. McGibbon, K. A. Beauchamp, M. P. Harrigan, C. Klein, J. M. Swails, C. X. Hernández, C. R. Schwantes, L.-P. Wang, T. J. Lane, V. S. Pande, MDTraj: A modern open library for the analysis of molecular dynamics trajectories, Biophysical Journal 109 (8) (2015) 1528–1532. doi:10.1016/j.bpj.2015.08.015.

[28] W. Humphrey, A. Dalke, K. Schulten, VMD: Visual molecular dynamics, Journal of Molecular Graphics 14 (1) (1996) 33–38. doi:10.1016/0263-7855(96)00018-5.

[29] R. Goswami, A. Goswami, J. K. Singh, d-SEAMS: Deferred structural elucidation analysis for molecular simulations, Journal of Chemical Information and Modeling 60 (4) (2020) 2169–2177. doi:10.1021/acs.jcim.0c00031.

[30] T. Meyer, C. Ferrer-Costa, A. Pérez, M. Rueda, A. Bidon-Chanal, F. J. Luque, C. A. Laughton, M. Orozco, Essential dynamics: A tool for efficient trajectory compression and management, Journal of Chemical Theory and Computation 2 (2) (2006) 251–258. doi:10.1021/ct050285b.

[31] E. Jakupovic, O. Beckstein, MPI-parallel molecular dynamics trajectory analysis with the H5MD format in the MDAnalysis Python package, in: Proceedings of the Python in Science Conference, SciPy, 2021, pp. 40–48. doi:10.25080/majora-1b6fd038-005.

[32] Open Geospatial Consortium, Zarr storage specification 2.0 community standard (2022). doi:10.62973/21-050r1.

[33] W. F. Godoy, N. Podhorszki, R. Wang, C. Atkins, G. Eisenhauer, J. Gu, P. Davis, J. Choi, K. Germaschewski, K. Huck, A. Huebl, M. Kim,

J. Kress, T. Kurc, Q. Liu, J. Logan, K. Mehta, G. Ostrouchov, M. Parashar, F. Poeschel, D. Pugmire, E. Suchyta, K. Takahashi, N. Thompson, S. Tsutsumi, L. Wan, M. Wolf, K. Wu, S. Klasky, ADIOS 2: The Adaptable Input Output System. A framework for high-performance data management, SoftwareX 12 (2020) 100561. doi:10.1016/j.softx.2020.100561.

[34] S. A. Brown, M. Folk, G. Goucher, R. Rew, P. F. Dubois, Software for portable scientific data management, Computers in Physics 7 (3) (1993) 304–308. doi:10.1063/1.4823180.

[35] L. Martínez, R. Andrade, E. G. Birgin, J. M. Martínez, PACKMOL: A package for building initial configurations for molecular dynamics simulations, Journal of Computational Chemistry 30 (13) (2009) 2157–2164. doi:10.1002/jcc.21224.

[36] G. A. Tribello, M. Bonomi, D. Branduardi, C. Camilloni, G. Bussi, PLUMED 2: New feathers for an old bird, Computer Physics Communications 185 (2) (2014) 604–613. doi:10.1016/j.cpc.2013.09.018.

[37] F. Weik, R. Weeber, K. Szuttor, K. Breitsprecher, J. de Graaf, M. Kuron, J. Landsgesell, H. Menke, D. Sean, C. Holm, ESPResSo 4.0 – an extensible software package for simulating soft matter systems, The European Physical Journal Special Topics 227 (14) (2019) 1789–1816. doi:10.1140/epjst/e2019-800186-9.

[38] M. Ceriotti, J. More, D. E. Manolopoulos, i-PI: A Python interface for ab initio path integral molecular dynamics simulations, Computer Physics Communications 185 (3) (2014) 1019–1026. doi:10.1016/j.cpc.2013.10.027.

[39] V. Kapil, M. Rossi, O. Marsalek, R. Petraglia, Y. Litman, T. Spura, B. Cheng, A. Cuzzocrea, R. H. Meißner, D. M. Wilkins, B. A. Helfrecht, P. Juda, S. P. Bienvenue, W. Fang, J. Kessler, I. Poltavsky, S. Vandenbrande, J. Wieme, C. Corminboeuf, T. D. Kühne, D. E. Manolopoulos, T. E. Markland, J. O. Richardson, A. Tkatchenko, G. A. Tribello, V. Van Speybroeck, M. Ceriotti, i-PI 2.0: A universal force engine for advanced molecular simulations, Computer Physics Communications 236 (2019) 214–223. doi:10.1016/j.cpc.2018.09.020.

[40] Y. Litman, V. Kapil, Y. M. Y. Feldman, D. Tisi, T. Begušić, K. Fidanyan, G. Fraux, J. Higer, M. Kellner, T. E. Li, E. S. Pós, E. Stocco, G. Trenins, B. Hirshberg, M. Rossi, M. Ceriotti, i-PI 3.0: A flexible and efficient framework for advanced atomistic simulations, The Journal of Chemical Physics 161 (6) (2024) 062504. doi:10.1063/5.0215869.

[41] T. A. Barnes, E. Marin-Rimoldi, S. Ellis, T. D. Crawford, The MolSSI Driver Interface Project: A framework for standardized, on-the-fly interoperability between computational molecular sciences codes, Computer Physics Communications 261 (2021) 107688. doi:10.1016/j.cpc.2020.107688.

[42] T. van Mourik, G. J. Harris, O. L. Polyansky, J. Tennyson, A. G. Császár, P. J. Knowles, Ab initio global potential, dipole, adiabatic, and relativistic correction surfaces for the HCN–HNC system, The Journal of Chemical Physics 115 (8) (2001) 3706–3718. doi:10.1063/1.1383586.

[43] T. Hoefler, R. Belli, Scientific benchmarking of parallel computing systems: twelve ways to tell the masses when reporting performance results, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, ACM, 2015, pp. 1–12. doi:10.1145/2807591.2807644.

[44] T. Kalibera, R. Jones, Rigorous benchmarking in reasonable time, in: Proceedings of the 2013 international symposium on memory management, ACM, 2013, pp. 63–74. doi:10.1145/2491894.2464160.

[45] H. B. Mann, D. R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, The Annals of Mathematical Statistics 18 (1) (1947) 50–60. doi:10.1214/aoms/1177730491.

[46] M. Hollander, D. A. Wolfe, E. Chicken, Nonparametric Statistical Methods, 3rd Edition, Wiley, Hoboken, NJ, 2014.

[47] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70. URL https://www.jstor.org/stable/4615733

[48] Bloomberg, memray: a memory profiler for Python, https://github.com/bloomberg/memray, version 1.20.0. Accessed 2026-10-31 (2026).

[49] N. A. Krylov, R. G. Efremov, libxtc: an efficient library for reading XTC-compressed MD trajectory data, BMC Research Notes 14 (1) (2021) 124. doi:10.1186/s13104-021-05536-5.

[50] M. Gunde, N. Salles, A. Hémeryck, L. Martin-Samos, IRA: A shape matching approach for recognition and comparison of generic atomic patterns, Journal of Chemical Information and Modeling 61 (11) (2021) 5446–5457. doi:10.1021/acs.jcim.1c00567.

[51] A. M. Smith, D. S. Katz, K. E. Niemeyer, Software citation principles, PeerJ Computer Science 2 (2016) e86. doi:10.7717/peerj-cs.86.

[52] F. Bigi, J. W. Abbott, P. Loche, A. Mazitov, D. Tisi, M. F. Langer, A. Goscinski, P. Pegolo, S. Chong, R. Goswami, P. Febrer, S. Chorna, M. Kellner, M. Ceriotti, G. Fraux, metatensor and metatomic : Foundational libraries for interoperable atomistic machine learning, The Journal of Chemical Physics (2026). doi:10.1063/5.0304911.
URL https://doi.org/10.1063/5.0304911